

SOUP_PLAN.md — FRTCON → SOUPCON rebrand & rebuild

This is a running plan for turning this fork of `frtcon.com` into **soupcon.org** (“**Soup Conditions**”) — worked one item at a time. Like `PLAN.md`, each item gets a **Done:** note when finished (what changed, how it was verified), so a later session doesn’t redo work or re-litigate a decision already made here. Unlike `PLAN.md`, this isn’t a code-review list — it’s a build-out plan for a genuinely new app concept, sharing this codebase’s plumbing (React/Vite, NWS API, caching, PWA setup) but not its classification logic or content.

`frtcon.com` is explicitly **out of scope** and untouched — it keeps existing as-is. `soupcon.org` is a new, independent deployment of this fork.

Decisions locked in (2026-09-27, per user)

- **Concept:** SOUPCON = “Soup Conditions.” Same 1=worst / 5=best shape as FRTCON, but about rain instead of winter storms:

Level	Meaning
1	Currently raining
2	Rain expected in the next 12 hours
3	Rain expected in the next 48 hours (but not the next 12)
4	Cloudy and damp, no rain expected soon
5	Clear and sunny for the next several days

- **Data source:** NWS hourly forecast (gridpoint forecast/hourly) drives the 12h/48h windows, current conditions (nearest station’s /observations/latest) drive “currently raining,” and the multi-day /forecast drives the level-5 “clear for days” outlook. The alerts endpoint (`getActiveAlertsByPoint`) is **not** part of classification — see the open question below on whether it’s kept for anything else.
- **Recipe modal:** replaced with a chicken noodle soup recipe for now. Rotating/multiple soup recipes are a real feature but explicitly “developed later” — don’t build a rotation mechanism as part of this plan, just swap in the one recipe (matches this project’s own “don’t build for hypothetical future requirements” norm).
- **Infra:** `soupcon.org` gets its own domain registration, DNS zone, and Apache vhost, set up the same way `frtcon.com` was (Cloudflare Tunnel, Apache origin, certbot+DNS-01) but **as a separate site**, not folded into the existing `frtcon.com` config/cert.

Open questions (resolve at the step that hits them, not now)

- **Raw alerts panel:** FRTCON shows every raw active NWS alert for the location alongside its score. Does SOUPCON keep an analogous “active NWS alerts here” panel (Flood Warning, Flood Advisory, etc. would still be genuinely relevant to a rain app), or drop it since the score itself is now forecast-based, not alert-based? Recommendation when we reach step 3: keep a lightweight version — flood-family alerts are on-theme — but this is a real product call, not a mechanical rename.
- **SnowOverlay.jsx:** falling-snow decoration doesn’t fit a rain theme. Likely becomes a falling-rain (or sun-rays-on-clear-days) equivalent. Decide the specific visual when we get to step 5.
- **Location label:** FRTCON shows the NWS forecast *zone* name (from `getZoneByPoint`). NWS’s `/points` response also returns `properties.relativeLocation` (city/state), which may read better for a “should I make soup” app than a zone name. Decide at step 2.
- **Nearest-station staleness:** `/observationStations` can return stations that haven’t reported in hours. Step 1/2 needs a strategy (try the first N stations, take the freshest) rather than blindly

using station #1 — flag as a real gotcha to test against live data, the way #1 in PLAN.md did for zone-vs-point alerts.

- **Visual theme:** FRTCON's palette (#0b1f3a deep blue, snow) is a winter palette. SOUPCON probably wants something rain/soup-coded (greys, warm broth tones?) — hold off until branding step (7), ask then if not obvious.
 - **Domain registration status:** confirm soupcon.org is actually registered before step 12 (infra) is actionable at all.
-

Sequenced work items

1. Core classification rewrite — src/lib/soupcon.js — **FIXED**

Replaces src/lib/frtcon.js (frtcon.js itself untouched for now — still used by App.jsx until item 5). Pure functions, no React/DOM dependency (same reason as before: directly unit-testable). Proposed shape:

- Input: hourly forecast periods (next ~48h), latest observation, and extended/daily forecast periods (next ~7 days) — not a list of alert features like classifyAlert took.
- `classifyForecast({ hourly, observation, extended })` → walks levels 1→5, first match wins (same ordered-checks pattern as `classifyAlert`):
 1. Observation's `textDescription/present-weather` indicates rain now.
 2. Any of the next 12 hourly periods' `shortForecast / probabilityOfPrecipitation` indicates rain.
 3. Any of hours 12–47 (but none of 0–11) indicate rain.
 4. Extended forecast reads `cloudy/overcast/damp/foggy` for the near term.
 5. Otherwise — extended forecast reads `clear/sunny` for the outlook window.
- Rain/clear keyword matching needs the same care `classifyAlert`'s comment gives NWS's event field: `shortForecast` is closer to free text than event was, so this needs real keyword lists (rain/showers/thunderstorms/drizzle vs. sunny/clear/mostly clear/mostly sunny) validated against live NWS output, not assumed — note day/night wording differs (“Sunny” vs. “Mostly Clear”).
- Write vitest tests alongside this from the start (see item 11) instead of shelving them the way PLAN.md #11 got shelved for FRTCON.
- **Done:** implemented in `src/lib/soupcon.js` as `classifySoupcon` (plus exported helpers `periodIndicatesRain/observationIndicatesRain` and a copied-over `pickRandomItems`), following the proposed shape above: ordered checks 1→5, first match wins. Rain detection matches keywords (rain/showers/shower/thunderstorm/drizzle/sprinkles) in `shortForecast/textDescription`, or a $\geq 40\%$ `probabilityOfPrecipitation`, or a positive `precipitationLastHour` reading. Level 1 prefers a real observation and only falls back to the current hourly period when no observation is available at all. Levels 2/3 split the hourly forecast at the 12-hour mark. Levels 4/5 look at the next 4 extended (12-hour) periods for `cloudy/damp` keywords, defaulting to level 5 (clear) when nothing matches — mirrors FRTCON's own “nothing else matched → level 5” default.
 - Added vitest as a dev dependency with a “test”: “vitest run” script (pinned to ^2.1.9, not the latest 5.x — see the Node-version note below) and `src/lib/soupcon.test.js` covering: each level via representative fake data, the 12-vs-48-hour boundary, the no-observation fallback for level 1, the “only the next 4 extended periods count” boundary for the 4-vs-5 split, and `pickRandomItems`'s existing `min(count, length)` unique-items behavior. 19 tests, all passing.
 - **Not done in this step, left for item 5:** deleting `frtcon.js` or wiring `soupcon.js` into `App.jsx` — this item is scoped to the pure classification module only, so the app still runs on the old FRTCON/alerts logic for now, unchanged and still buildable.
 - **Environment note:** this shell's system Node (18.19.1) can't actually run vitest/vite build at all — Vite 8 needs Node 20.19+/22.12+, and a separate npm-9.x optional-dependency bug broke `node_modules` further until reinstalled under a newer npm. Verified `npm run test`, `npm run lint`, and `npm run build` all pass, but only after installing `node_modules` and running them with a Node 20+ binary (a scratch-space download, not a system upgrade — see the

new `CONTEXT.md` gotcha for detail). No production/deploy environment is assumed to have this problem; it's specific to this dev shell as currently set up.

2. New API layer — `src/lib/weatherApi.js` — **FIXED**

- `getHourlyForecast(lat, lon, opts)` — `/points` → `gridpointURL` → `/gridpoints/{office}/{x},{y}/forecast/hourly`
- `getCurrentConditions(lat, lon, opts)` — `/points` → `properties.observationStations` → try stations in order until one has a recent `/observations/latest`.
- `getExtendedForecast(lat, lon, opts)` — same gridpoint's `/forecast` (12-hour periods, ~7 days).
- Decide here: keep `getZoneByPoint` (for a label) or switch to `relativeLocation` from the `/points` response — see open question above.
- Decide here: keep or drop `getActiveAlertsByPoint` per the open question above.
- Cache TTLs: observations should be short (comparable to today's 5-minute `ALERTS_CACHE_TTL_MS`); hourly forecast can be somewhat longer (NWS updates it roughly hourly); extended forecast longer still. New prefixes in `cache.js` (see item 9).
- **Done:** added three new exports to `weatherApi.js` — `getHourlyForecast`, `getExtendedForecast`, `getCurrentConditions` — plus a private, shared `getGridpointInfo(lat, lon)` helper that all three (and only they) use, so a single `/points` call/cache entry serves all three instead of each hitting `/points` separately for the same location.
 - **Location label decided:** `getGridpointInfo` reads `properties.relativeLocation.properties.city` off the same `/points` response and returns it as `locationLabel` (e.g. "Seattle, WA") — confirmed live against `api.weather.gov/points/47.6062,-122.3321`. Chosen over the FRTCON-era forecast-zone name per the open question above. `getZoneByPoint/getActiveAlertsByPoint` are untouched and still exported — item 3 decides whether either is still used.
 - **Nearest-station staleness handled:** a new private `findFreshObservation` tries up to 5 stations (in the order NWS's `/gridpoints/{office}/{x},{y}/stations` collection returns them) and takes the first with an observation timestamp within `OBSERVATION_MAX_AGE_MS` (90 minutes), rather than trusting station #1 blindly. `getCurrentConditions` returns null (not a throw) if none qualify, or if the stations request itself fails — it's a best-effort secondary source and `classifySoupcon` already knows how to fall back to the current hourly period when observation is null.
 - **Cache TTLs added to `cache.js`:** `GRIDPOINT_CACHE_TTL_MS` (1h, same as the old zone cache), `HOURLY_FORECAST_CACHE_TTL_MS` (30m), `EXTENDED_FORECAST_CACHE_TTL_MS` (2h), `OBSERVATION_CACHE_TTL_MS` (5m, matching the existing alerts TTL) — with matching `GRIDPOINT_CACHE_PREFIX/HOURLY_FORECAST_CACHE_PREFIX/EXTENDED_FORECAST_CACHE_PREFIX/OBSERVATION_CACHE_PREFIX` keys (still `frtcon_`-namespaced for now — see item 9) and cache-key builders following the existing `makeZoneCacheKey/makeAlertsCacheKey` pattern. All four new prefixes were also added to `sweepExpiredCache's TTL_MS_BY_PREFIX` map so they get swept the same way existing keys do. A `getCurrentConditions` miss (no fresh station found) is deliberately *not* cached, since `getCacheItem` can't distinguish a cached null from a genuine cache miss anyway (see `cache.js`) — no behavior lost by skipping the write.
 - **Verified against live `api.weather.gov`** (not just unit tests, since this item is almost entirely about matching real API shapes): fetched real `/points`, `/forecast/hourly`, `/forecast`, `/stations`, and `/observations/latest` responses for Seattle, WA and confirmed the exact field names/shapes this code assumes (`forecastHourly/forecast/observationStations/relativeLocation` URLs and fields, `stations.features[].id` being a full observation URL, `observations/latest's timestamp/textDescription/precipitationLastHour` shape). Separately pulled 48h of hourly periods plus 6 extended periods across 5 rain-prone cities (Miami, New Orleans, Houston, Orlando, Tampa) and collected every distinct `shortForecast` string seen (Chance Showers And Thunderstorms, Slight Chance Rain Showers, Showers And Thunderstorms Likely then Chance Showers And Thunderstorms, plus the clear/cloudy variants) — confirmed all of them are caught by `soupcon.js's` existing `RAIN_KEYWORDS/CLOUDY_KEYWORDS/CLEAR_KEYWORDS` lists with no surprises, so no changes were needed there. `npm run lint`, `npm run test` (19/19 still passing), and `npm run build` all pass.

3. Wire into App.jsx — FIXED

- Replace the `frtccon/frtcconMessage` useMemos with `soupcon/ soupconMessage`, fed by the new data instead of `result.alerts`.
- `result` shape grows to carry hourly/observation/extended data instead of (or alongside, if the alerts panel is kept) alerts.
- The refresh machinery (`fetchAt`, `STALE_ON_VISIBLE_MS`, `visibilitychange` effect, auto-refresh interval) is data-shape-agnostic and should carry over largely unchanged.
- Resolve the “raw alerts panel” open question here, since it determines whether `AlertCard.jsx` and the alerts fetch stay in the tree at all.
- **Done:**
 - **Alerts panel:** kept, decoupled from the score, per the plan’s own recommendation. `result.alerts` is still fetched (`getActiveAlertsByPoint`) and rendered via `AlertCard`; the count tag next to the badge is unchanged. The one thing genuinely removed is the “Winter alerts driving the score” block - `classifySoupcon` has no `matchingAlerts` equivalent (the score isn’t alert-derived anymore), so that JSX block is gone, replaced with a comment explaining why.
 - **result shape:** now `{ source, lat, lon, locationLabel, hourlyPeriods, extendedPeriods, observation, alerts, fetchAt }`. `zone` is gone; `getZoneByPoint` (and the now-orphaned `extractZoneIdFromUrl`, `makeZoneCacheKey`) were deleted from `weatherApi.js/cache.js` as confirmed-dead code once this was its only call site - `ZONE_CACHE_PREFIX` itself was kept (just unused for new writes) so `sweepExpiredCache` can still clean up any leftover `frtccon_zone_lookup_*` keys from before this change.
 - **runLookupFromCoordinates** now fetches `getLocationLabel`, `getHourlyForecast`, `getExtendedForecast`, `getCurrentConditions`, and `getActiveAlertsByPoint` in parallel via `Promise.all` with the shared signal - the first four collapse into a single `/points` request in practice via `getGridpointInfo`’s new in-flight de-dup (added in this step, not item 2, once the concurrent-call pattern this creates became concrete).
 - **Refresh machinery:** `refreshAlerts` generalized to `refreshWeatherData`, refetching all four SOUPCON-relevant sources (`skipCache: true`) on the same interval/`visibilitychange` triggers as before - deliberately not staggering hourly/extended forecast to their own longer TTLs here, to keep this step’s scope to wiring rather than a new tiered-refresh scheduler; those TTLs still do real work for a repeat lookup of the same location within the cache window. Flagging in case the extra request volume (4x vs. the old 1x every 5 minutes) is worth revisiting later.
 - **handleShare:** updated to the new variable names and `result.locationLabel` in place of `result.zone.zoneName`. Left otherwise untouched - it still says “French Toast Condition” and still shares `frtccon.com`’s URL, which is content/domain work (items 4/7/8), not plumbing. Noting explicitly: this call site’s hardcoded `frtccon.com` wasn’t named in items 7/8’s file lists (`index.html`, `manifest.json`, `robots.txt`, `sitemap.xml`, `index.md`) - flagging it here so it doesn’t get missed when those items land.
 - **Known, deliberate intermediate-state rough edges** (all scoped to later items, not oversights): the badge still literally renders the text “FRTC CON” (hardcoded inside `FrtcconBadge.jsx`, not a prop - item 5), `FrtcconMessage` still says “French Toast Condition #N” (hardcoded - item 5), the condition box’s headline/title/commentary are still French-toast-flavored (`alertMessages.js` - item 4), CSS classes are still `.frtccon-*` (item 6), and `SnowOverlay` still renders falling snow rather than rain (item 5). The *data* driving all of this - `soupcon.level`, `soupcon.title`, `soupcon.reason`, the location label - is correctly SOUPCON-based as of this step; only the surrounding text/components haven’t caught up yet.
 - **Verification:** `npm run lint`, `npm run test` (19/19), and `npm run build` all pass. Also smoke-tested that `npm run dev` serves the app without a compile/transform error (confirmed `App.jsx` and its new imports resolve correctly through Vite’s dev server). **Not verified live in an actual browser** - no browser automation tool was available this session (Claude in Chrome wasn’t connected). This is a meaningful gap for a step this size (real ZIP/geolocation lookups, the parallel fetch, the refresh effects, and the share button are all exercised for the first time here) - worth a real interactive pass before treating this as fully confirmed, the way

earlier PLAN.md items were.

4. Content rewrite — **FIXED**

- New `src/data/soupMessages.js` (replaces `alertMessages.js`): 5 levels, soup/rain-themed headline + title + rotating commentary lines, same shape `pickRandomItems` already consumes.
- `src/data/recipe.js` → chicken noodle soup recipe (per decision above).
- Copy pass over hardcoded strings in `App.jsx` (Share button text, headings like “Active Zone Alerts”, aria-labels mentioning FRTCON).
- **Done:**
 - **`src/data/soupMessages.js`** created with 5 levels matching the locked-in SOUPCON scale (currently raining / rain in 12h / rain in 48h / cloudy-damp / clear-sunny), each with a `#SOUPCONn` `CONDITION` headline, a short title, and 20 rotating commentary lines mirroring the old FRTCON copy’s joke density and structure but reworked around rain/soup imagery (broth, stockpots, umbrellas, gutters, egg noodles) instead of French toast/winter imagery. Same `{headline, title, body}` shape as before, so `pickRandomItems`/the `App.jsx` consumption code needed no changes beyond the import swap.
 - **`alertMessages.js` deleted** (not just superseded) once confirmed to have no other consumers — `soupMessages.js` fully replaces it, unlike `frtccon.js` which item 1 deliberately left in place until item 5.
 - **`src/data/recipe.js`** replaced with a standard chicken noodle soup recipe (per decision above — rotating soup recipes are explicitly future work, not built here).
 - **`RecipeModal.jsx` cleanup found along the way:** the component had a “Wait.”-prefix bold-emphasis special case written specifically for the old French toast recipe’s phrasing (“Wait. You’ll be tempted to flip too soon...”). The new recipe has no step matching that pattern, making the regex dead weight rather than a latent feature, so it was removed in favor of plain step rendering — a direct consequence of this item’s content swap, not a tangential refactor, so handled here rather than deferred to item 5 (which only expected recipe *title* text to change, not this).
 - **Copy pass in `App.jsx`:** `<h1>` (“What’s my Soup Condition?”), the subtitle (“Check the rain outlook for your area and see your current Soup Condition.”), the dropdown menu item (“Soup Recipe”), the share text’s “Soup Condition #N” line, and both “copy your ... status” toast messages (now “SOUPCON status”). Confirmed via `grep` that no FRTCON/French Toast text remains anywhere in `App.jsx` – what’s left there (`frtccon_*` `localStorage` keys, `.frtccon-*` CSS classes, `frtccon.com`’s share URL) is exactly and only the items 6/7/8/9 territory called out in earlier Done notes.
 - `npm run lint`, `npm run test` (19/19), and `npm run build` all pass.
 - **Not verified live in a browser** (same gap as item 3 — Claude in Chrome isn’t connected this session): the recipe modal’s rendering and print view, and the on-screen condition box’s new copy, haven’t been visually confirmed, only reviewed as code/JSX.

5. Component/file renames — **FIXED**

- `src/lib/frtccon.js` → `src/lib/soupcon.js` (already done in item 1).
- `FrtcconBadge.jsx` → `SoupconBadge.jsx`, `FrtcconMessage.jsx` → `SoupconMessage.jsx` — mechanical renames + prop names.
- `RecipeModal.jsx` — title text, no structural change expected.
- Resolve `SnowOverlay.jsx`’s fate (open question above) and `AlertCard.jsx`’s fate (tied to item 3’s decision).
- **Done:**
 - **`frtccon.js` deleted.** Confirmed zero remaining importers (item 3 already stopped using it); `soupcon.js`’s own comment about the copied-not-imported `pickRandomItems` updated to reflect that it’s now the only copy, not “still shared until item 5.”
 - **`FrtcconBadge.jsx` → `SoupconBadge.jsx`, `FrtcconMessage.jsx` → `SoupconMessage.jsx`:** re-named exports, hardcoded label text fixed (“FRTCON N” → “SOUPCON N”; “French Toast Condition #N” → “Soup Condition #N”), and the `zoneName` prop renamed to `locationLabel` to match what `App.jsx` actually passes (a mismatch left over from item 3’s data plumbing,

- cleaned up now that this is the component-renames step). SoupconMessage's footnote – which specifically referenced “the French Toast Alert System” (a real thing FRTCON was parodying) – was rewritten rather than find-replaced, since that sentence describes a completely different concept for SOUPCON: now reads “* #SOUPCON only estimates whether rain is likely to disrupt your day where you live. It does not measure total rainfall, storm severity, or anything outside this 5-level rain-likelihood scale.” CSS class names on both (.frtcon-badge, .frtcon-condition-status, etc.) deliberately left untouched – that's item 6, not this one.
- **SnowOverlay.jsx → RainOverlay.jsx** (resolves the open question): falling snowflake glyphs replaced with falling rain-streak s (thin gradient lines, faster/straighter fall, no horizontal drift – rain doesn't sway side-to-side the way snow does). .snow-overlay → .rain-overlay and @keyframes snowFall → @keyframes rainFall in styles.css (a required part of this rename, not deferred to item 6, since the old class/keyframe would otherwise be orphaned dead CSS the moment the component using them was renamed).
 - * **Density mapping also revisited, not just the visual:** FRTCON showed snow at every level, even level 5 (“all clear”), as a year-round mascot effect independent of severity. For SOUPCON, showing rain falling on screen when the score explicitly says “no rain expected” (levels 4/5) would contradict the app's own message, so those two levels now show zero rain rather than reusing FRTCON's old “always show at least a little something” floor of 8. Levels 1-3 keep a declining-but-nonzero mapping (140/90/40) reflecting decreasing rain urgency. Renamed snowCount → rainCount accordingly.
 - **AlertCard.jsx:** no changes – already resolved in item 3 (kept, decoupled from the score), and it has no FRTCON-specific naming to rename in the first place.
 - **RecipeModal.jsx:** no further changes needed here specifically (its “Wait.” special-case cleanup already happened in item 4, as a direct consequence of the recipe content swap); title text already flows through RECIPE.title automatically.
 - Confirmed via repo-wide grep: no remaining FrtconBadge/FrtconMessage/SnowOverlay/snowCount/lib references anywhere. Remaining FRTCON/French Toast hits are all in code comments (historical “unlike FRTCON...” explanations, fine to keep indefinitely) or in styles.css comments describing still-.frtcon-* -named CSS (item 6) – nothing user-facing left unaddressed.
 - npm run lint, npm run test (19/19), and npm run build all pass.
 - **Not verified live in a browser** (same gap as items 3/4 – Claude in Chrome still isn't connected this session): the rain visual itself (density, speed, whether it reads as rain rather than noise) is reviewed as code/CSS only, not seen rendered.

6. CSS renames — src/styles.css — **FIXED**

- .frtcon-* → .soupcon-* classes (.frtcon-badge--level-N, .frtcon-condition-status, .frtcon-status-row, .frtcon-updated-at, .frtcon-matching-alerts*, .frtcon-title-large, etc.).
- Verify visually in both light/dark and at phone width after, same as PLAN.md #9's verification approach.
- **Done:**
 - Every .frtcon-* class/id in src/ (styles.css and the JSX that references them: App.jsx, SoupconBadge.jsx, SoupconMessage.jsx, RecipeModal.jsx) renamed to .soupcon-*, including the frtcon-zip-input DOM id (label/input pair) alongside the class renames, since it's the same category of internal branding-prefixed identifier even though it's technically an id, not a class.
 - **.frtcon-matching-alerts/.frtcon-matching-alerts-title deleted outright, not renamed** — exactly the dead CSS the plan's own example list flagged (.frtcon-matching-alerts*), confirmed via grep to have zero remaining JSX consumers since item 3 removed the “Winter alerts driving the score” block. CSS bundle shrank slightly (7.42 kB → 7.33 kB) consistent with real dead-rule removal, not just a rename.
 - Section-header comments in styles.css that named “FRTCON” (e.g. “The beige/amber box containing the FRTCON headline...”) updated to “SOUPCON” alongside the class renames in the same section.
 - Repo-wide grep confirms zero remaining frtcon- (hyphenated) hits anywhere in src/; all

frtcon hits left are the already-known, already-deferred ones: `frtcon_last_*/frtcon_*_lookup_/etc.` (underscore-separated storage keys, item 9) and `frtcon.com` (domain, items 7/8).

- **New scope item found while checking, not yet acted on:** `public/sw.js` has its own independent set of `frtcon-/frtcon_-`prefixed identifiers — DOM ids `frtcon-reconnect-text/frtcon-reconnect-retry` and a `frtcon_reconnect_attempts` `sessionStorage` key used by the offline “Reconnecting...” fallback page (see `CONTEXT.md`’s SW notes) — plus `src/main.jsx` clears that same `sessionStorage` key on a successful load. None of this is `src/styles.css` or JSX class names, so it’s out of this item’s scope, and it wasn’t explicitly named in items 7/8/9’s file lists either. Flagging here (as with the Share-button URL gap found in item 3) so it isn’t missed when those items land — likely belongs with item 9 (storage/id key renames) rather than 7/8 (which are about domain references specifically), but worth double-checking then.
- `npm run lint`, `npm run test` (19/19), and `npm run build` all pass.
- **Not verified live in a browser** (same recurring gap — Claude in Chrome still isn’t connected this session): light/dark mode, phone width, both modals, and the recipe print preview are all unverified visually; a class-name-only rename (no selector structure or property changes) is low-risk, but this is exactly the kind of change `PLAN.md`’s own #9 treated as warranting a real before/after visual pass, not just a successful build.

7. Branding/meta pass — **FIXED**

- `package.json` name (`frtcon_weather_alert_app` → something like `soupcon_weather_app`).
- `index.html`: `<title>`, meta description, OG tags, `og:image/og:url` domain refs → `soupcon.org`.
- `public/manifest.json`: name, short_name, description.
- `icons/favicon`: current artwork (snowflake-on-blue) doesn’t fit — needs new icon assets (192/512/512-maskable + favicon). Flag to the user rather than inventing brand artwork unilaterally.
- `theme-color/background_color` — pick together with the visual palette open question above.
- **Done:**
 - **Icons (asked, per decision above):** user specified the design directly — “a simple two tone bowl (oval on top of an upwards facing semicircle) in dark and light purple under some blue raindrops, all on a lavender background.” First pass: hand-authored SVG (a full circle with its top half erased by a same-color rect, leaving a semicircle “bowl body”; an ellipse “rim”; three bezier teardrop raindrops above), rasterized via `rsvg-convert/ImageMagick` into all required sizes. User feedback on that pass: “weren’t bold enough.” **Regenerated from a user-provided master** (`/tmp/favicon.svg`, edited in Inkscape from the first pass) with a larger bowl/rim (1.667x) and larger raindrops (2.5x), more saturated colors (`#a28ead` bg, `#9042ad` bowl, `#602c74` rim, `#5b8def` drops unchanged), and content shifted upward for better balance. From that single master:
 - * `favicon.svg/icon-192.png/icon-512.png` — the master as-is (already rounded-square clipped).
 - * `apple-touch-icon.png` — same design with the rounded-square clip-path stripped (full-bleed square, no transparency, matching Apple’s own convention of applying its own corner rounding).
 - * `icon-512-maskable.png` — full-bleed square with the bowl/drops group scaled 0.85 and centered. Checked the math (not just eyeballed this time): the bold master’s furthest extremes (bowl bottom tip, top raindrop tip) sit right at ~210px from center against a ~205px safe radius for an 80%-safe-zone circular mask – i.e. the fully bold version was *just* outside safe for a strict circular crop, so the maskable variant specifically gets a modest 0.85 scale-down (much less conservative than the first pass’s 0.72) rather than reusing the bold version unscaled or over-shrinking it again.
 - * `favicon.ico` — multi-res 16/32/48 from the same master. Checked all sizes visually (512/192/64px) before finalizing – reads as bold and legible even at favicon size.
 - **Color palette (asked, “switch to a new rain-coded palette”):** proposed and applied keeping the *blue* accent family (buttons, status-box border/text, alert chips) unchanged – it ties directly to the icon’s raindrops, so “purple shell + blue rain accents” mirrors the icon exactly rather than going all-purple. Converted only the navy “chrome” colors (page/card/modal back-

grounds, borders, dropdown background, input background, dark-text-on-light-button) to a purple equivalent at matching lightness, e.g. #0b1f3a→#1a0f2e (darkest bg), #122b4d→#2d1b4a (card bg), #27466f→#4a3270 (border) – full mapping across `src/styles.css`, `index.html`'s theme-color, `manifest.json`'s theme/background colors, and `public/sw.js`'s fallback page (same app, needed the same palette). Deliberately left unchanged: the Facebook-brand blue share button, the red error box, the amber condition-status callout box, and the five severity-scale badge colors – none of those are “the app’s chrome,” they’re semantic/brand colors independent of the SOUPCON hue.

- `package.json` name, `index.html` title/description/OG tags (text, separate from the theme-color/palette work above), and `manifest.json` name/short_name/description all updated to SOUPCON copy and the `soupcon.org` domain.
- `npm run lint`, `npm run test` (22/22), and `npm run build` all pass; confirmed the new icons land in `dist/` and `manifest.json/` `package.json` are still valid JSON.
- **Not verified live in a browser:** the icons/palette are reviewed as rendered PNGs (via the icon-generation step itself) and code, but the actual running app’s look – installed PWA icon, theme-color browser chrome tinting, overall in-app color harmony – hasn’t been seen live, same recurring gap as items 3-6.

8. Domain/crawler files — FIXED

- `public/robots.txt`, `public/sitemap.xml`, `public/index.md`: `frtcon.com` → `soupcon.org` URLs.
- `.htaccess` (if any domain-specific values are in it — check when this step is reached).
- **Done:**
 - `robots.txt`'s Sitemap: line and `sitemap.xml`'s `<loc>` both updated to `soupcon.org`.
 - `.htaccess` checked — no domain-specific values in it at all (just relative rewrite rules and headers), so no change needed.
 - `index.md` fully rewritten, not just domain-swapped – it’s a markdown mirror of the app’s own description (served for Accept: `text/markdown` requests), so leaving FRTCON-era wording there while everything else changed would be a glaring, publicly-visible inconsistency. Now describes the SOUPCON scale and its actual data sources (hourly forecast, extended forecast, current observations – not “active alerts”).
 - **Also fixed while here, found as a gap in item 6’s done note:** the Share button’s Facebook `sharer.php?u=` URL and its surrounding comments in `App.jsx` (`handleShare`) still said `frtcon.com` – not explicitly named in this item’s file list, but clearly domain-scope work, so folded in now rather than left for later.
 - `npm run lint`, `npm run test` (22/22), and `npm run build` all pass.

9. localStorage key prefixes — `src/lib/cache.js` — FIXED

- `frtcon_zip_lookup_` / `frtcon_zone_lookup_` / `frtcon_alerts_` → `soupcon_*` equivalents, plus new prefixes for hourly/observation/extended caches from item 2. `frtcon_last_source` / `frtcon_last_zip` → `soupcon_*`.
- No migration shim needed — `soupcon.org` is a different origin, so old `frtcon_*` keys never exist there in the first place.
- **Done:**
 - All seven prefixes in `cache.js` renamed (`ZIP_CACHE_PREFIX`/`ZONE_CACHE_PREFIX`/`ALERTS_CACHE_PREFIX`/`GRIDPOINT_CACHE_PREFIX`/`HOURLY_FORECAST_CACHE_PREFIX`/`EXTENDED_FORECAST_CACHE_PREFIX`/`OBSERVATION_CACHE_PREFIX`) plus the stale comments explaining the temporary `frtcon_`-namespacing (no longer applicable) cleaned up.
 - `App.jsx`'s `soupcon_last_source/soupcon_last_zip` (both read and write call sites, plus a comment) renamed from `frtcon_last_*`.
 - **Also handled the gap found in item 6’s done note**, since it belongs here: `src/main.jsx`'s `sessionStorage.removeItem(...)` call and `public/sw.js`'s own `STORAGE_KEY` constant, both now `soupcon_reconnect_attempts` (they share the same key across two files, so both needed updating together, not just one). Also renamed `sw.js`'s `frtcon-reconnect-text/frtcon-reconnect-retry` DOM ids (not strictly a storage key, but the same category

- of internal branding-prefixed identifier, found in the same file) and fixed its hardcoded `<title>Soup Conditions</title>` and “Can’t reach SOUPCON right now” fallback-page text, plus a top-of-file comment describing what the app shows.
- No migration code added, per the plan’s own reasoning – confirmed still applicable: `soupcon.org` is a new origin, so no `frtcon_*` keys exist there to migrate from.
- Repo-wide `grep` confirms zero remaining `frtcon/FRTCON/French Toast` hits anywhere in `src/`, `public/`, `index.html`, or `package.json` except historical code comments explaining past decisions (e.g. “Unlike FRTCON...”, “FRTCON-era `getZoneByPoint` name”) – all intentionally kept, matching this project’s own established comment style.
- `npm run lint`, `npm run test` (22/22), and `npm run build` all pass.
- **Not verified live in a browser:** the reconnect-fallback page (only reachable via a real navigation failure) and the ZIP/source auto-resume behavior are reviewed as code only – same recurring gap as items 3-7.

10. Docs rewrite — README.md and CONTEXT.md — **FIXED**

- Full rewrite once the app itself reflects the new concept (do this after items 1-9, not before, so it documents what’s actually true): what it does, the SOUPCON scale table, the three-endpoint data source rationale, project structure, deployment notes.
- `CONTEXT.md`’s `infra` section gets `soupcon.org`’s real `infra` facts once item 12 is done.
- Old `PLAN.md`’s still-open item (#11, classification tests) becomes moot once `frtcon.js` is replaced — note in `PLAN.md` that it’s superseded by this plan rather than leaving it looking abandoned. Decide then whether `PLAN.md` itself gets archived/retired for this fork.
- **Done:**
 - **README.md** fully rewritten: SOUPCON scale table, the forecast/observation classification rationale (with its own caveat about keyword-matching free-text `shortForecast`, mirroring how the old README caveated event-matching), updated project structure (`soupcon.js/soupcon.test.js`, `soupMessages.js`, `RainOverlay/SoupconBadge/SoupconMessage`), `npm run test` added to “Getting started,” the new `#1a0f2e` theme-color in the PWA `<head>` snippet, “APIs used” reworded to explain alerts are now independent of the score, a new limitations bullet about keyword-matching coverage, and “Ideas for later” updated (rotating soup recipes added; the affiliate idea reworded from `winter-gear/French-toast groceries` to `soup-ingredients/rainy-day gear`).
 - **CONTEXT.md** fully rewritten: new “What this project is” explains the fork relationship to `frtcon.com` explicitly (untouched, separate site) and points to `SOUP_PLAN.md`’s own “Decisions locked in”/Done notes the same way this file’s own Done notes matter. Infrastructure section deliberately **not** filled in with invented `soupcon.org` specifics — replaced with a “not yet set up, here’s the planned shape” note per the plan’s own instruction, explicitly warning not to assume `frtcon.com`’s specific facts (tunnel ID, vhost path, cert SANs) apply here. Frontend architecture section rewritten around the real new decisions: the forecast/observation-vs-alerts classification choice, station-staleness handling, the gridpoint in-flight de-dup, the location-label change, the decoupled alerts panel, and the icon/ palette decisions (new subsection, since those were real owner-directed decisions worth recording the same way the old Share button decisions were). Gotchas/decided-against/shelved sections carried over with FRTCON-specific wording fixed (e.g. the cloudflared zone-scoping gotcha now explicitly flags it as relevant to this fork’s own upcoming item-12 work) and one new shelved item added (rotating soup recipes, cross-referenced to item 4’s decision).
 - **PLAN.md:** added a banner note at the top marking the whole document superseded/historical (kept, not deleted or rewritten in place, since its own Done notes are still an accurate record of real FRTCON-era decisions) and specifically annotated item #11 (classification tests) as superseded by `soupcon.test.js` rather than left looking like an abandoned open item.
 - **CLAUDE.md updated too** (not explicitly named in this item, but a real gap found while doing this work): its session-bootstrap list only mentioned `PLAN.md`, with no pointer to `SOUP_PLAN.md` at all – a new session could easily have missed this plan entirely. Reworded the `PLAN.md` bullet to reflect its now-historical status and added a fourth bullet for `SOUP_PLAN.md`, plus updated “Keeping these in sync” to reference `SOUP_PLAN.md` as the doc that gets new Done notes going forward.

- Verified every file path named in the new README/CONTEXT actually exists (soupcon.js, soupcon.test.js, soupMessages.js, RainOverlay.jsx, SoupconBadge.jsx, SoupconMessage.jsx, etc.) and that no unintentional frtcon.com/FRTCON references remain (the ones that do remain in CONTEXT.md are deliberate: describing the sibling site's lineage and shared infra history). npm run lint, npm run test (22/22), and npm run build all still pass (docs-only change, but verified anyway per this project's own norms).

11. Tests — vitest — **FIXED**

- Add as dev dependency with a test script (same ask as old PLAN.md #11, but done this time as part of the rewrite instead of shelved).
- Cover soupcon.js: each level's representative forecast/observation shapes, the 12h-vs-48h boundary, day/night wording variants, and pickRandomItems (already covered, carries over unchanged).
- **Done:** vitest + the test script, and most of this coverage, were already added in item 1 (19 tests: each level, the 12h/48h boundary, the no-observation level-1 fallback, the 4-vs-5 extended-period window boundary, pickRandomItems). The one thing item 1 didn't yet have — **day/night wording variants** — added here as a new describe block in src/lib/soupcon.test.js:
 - Confirms daytime "Sunny"/"Mostly Sunny" and nighttime "Clear"/"Mostly Clear" both land on level 5, and "Partly Cloudy"/"Mostly Cloudy" both land on level 4 — i.e. the classifier doesn't only recognize daytime phrasing.
 - Also locks in the exact real shortForecast strings pulled from live api.weather.gov data during item 2's verification (including NWS's combined "X then Y" multi-condition phrasing, e.g. "Showers And Thunderstorms Likely then Chance Showers And Thunderstorms") as permanent regression tests, rather than leaving that verification as a one-off manual check that could silently regress later.
 - 22/22 tests passing; npm run lint and npm run build still pass.

12. Infra — soupcon.org — **FIXED (done by the owner directly)**

- Confirm domain registration (open question above).
- Cloudflare: new DNS zone for soupcon.org, nameservers pointed at Cloudflare, new ingress rule in the existing Tunnel's config.yml routing soupcon.org/www.soupcon.org to Apache — same tunnel, additional hostnames, not a new tunnel.
- Apache: new vhost /etc/apache2/sites-available/soupcon.conf, confirm which server/user it's served from (this repo already lives under /home/soupcon/, mirroring frtcon's /home/frtcon/public_html pattern — confirm that's the intended origin host before assuming it's the same physical server as elephant). Include a dev subpath mirroring frtcon.com's review-before-promote setup.
- TLS: certbot + certbot-dns-cloudflare, a cert covering soupcon.org + www.soupcon.org — a **separate** cert from frtcon.com's (unrelated site), not an --expand of the existing one.
- File permissions: world-readable (644), correct owner — this already bit FRTCON's PWA install flow once (see CONTEXT.md gotchas); check it explicitly during this step rather than after something silently fails.
- **Done:** set up directly by the owner (not Claude) — same Tunnel (dd742d1d-...), new ingress rule routing soupcon.org/www.soupcon.org/soupcon.thoughtleap.com to a new loopback port (127.0.0.1:8082), new vhost at soupcon.conf serving from /home/soupcon/soupcon.org/ with a dev/ subdirectory.
 - **Deviated from this item's own assumption, correctly:** no certbot cert was set up, and none is needed — the actual ingress rule (checked directly, for both soupcon.org and frtcon.com) speaks plain HTTP to the origin on all three loopback ports, not HTTPS-to-:443 as this item and CONTEXT.md's pre-rebuild wording assumed. Since nothing is publicly exposed except through the Tunnel, and Cloudflare's edge terminates the real browser-facing TLS regardless, an origin cert would add maintenance for no security benefit here. CONTEXT.md rewritten to reflect this (and to correct its own earlier assumption about frtcon.com's setup, which turned out to describe a config that doesn't match the real config.yml).
 - Verified live end-to-end (not just config review): soupcon.org, www.soupcon.org, and soupcon.org/dev/ all return real 200s through Cloudflare; file permissions checked recur-

sively, all correctly world-readable; title/theme-color/content on the live site match the current build.

- **Found and fixed along the way:** `mod_headers` wasn't enabled on this origin's Apache, silently dropping the Link discovery headers (`.htaccess`'s own `<IfModule>` guard degraded gracefully rather than erroring, exactly as designed, just not as *wanted* here) — this turned out to be a **pre-existing gap affecting frtcon.com identically**, not something this rollout introduced. Fixed with `a2enmod headers + reload` (done by the owner); confirmed both sites now send the header correctly.
- **Found, not yet resolved:** Cloudflare's "Managed robots.txt" is active on this zone and prepends its own content ahead of this app's own `robots.txt`, likely shadowing the explicit `ai-input=yes` stance — see `CONTEXT.md`'s Agent readiness/Infrastructure sections. Left as-is pending a decision on whether that matters enough to disable it for this zone.

13. Cutover/launch checklist — **FIXED (done by the owner directly)**

- Build, deploy to the dev path first, verify there (remembering `CONTEXT.md`'s note that PWA/install behavior can't be meaningfully tested from `/dev/`, only from the promoted root).
- Promote to `soupcon.org` root, confirm PWA install flow, confirm `robots.txt/sitemap.xml` are being served and match the agent-readiness setup FRTCON already did.
- **Done:** live at both the dev/ path and the promoted root, confirmed via direct HTTP checks against the real site (see item 12's Done note). `robots.txt/sitemap.xml` are served, though `robots.txt`'s content is currently modified in-flight by Cloudflare's managed `robots.txt` feature (see above) — not a deployment bug, a platform-level behavior to separately decide on. **Not yet verified:** the actual PWA install flow (Chrome's `beforeinstallprompt` eligibility, home-screen icon, standalone launch) — everything checked so far has been HTTP-level (`curl`), not a real browser/device install, which is the one thing this app's own `CONTEXT.md` says can only be meaningfully tested from the promoted root, not `/dev/` — worth doing before calling this fully launched.

14. Rain-vs-snow classification and animation — **FIXED**

Raised as a question after the plan's original 13 items were all done: "what happens if it snows instead of rains? should we be marking all precipitation events the same?"

- **Bug found while investigating, not just a design gap:** tested directly against the real `classifySoupcon` and confirmed a plain "Snow" forecast/observation with no reported `probabilityOfPrecipitation` matched none of the existing rain/cloudy/clear keyword lists and fell through to `SOUPCON5` "Clear and sunny" — the worst possible wrong answer, while it was actively snowing in the test data. "Snow Showers" was separately (also wrongly) swallowed into the rain bucket via the "showers" substring. Neither was a "we haven't gotten to this yet" gap; both were live, silent misclassifications.
- **Decision (per user):** word rain and snow separately, and swap in a snow-falling animation when snow is the primary forecast type, rather than either (a) treating all precipitation identically with one generic wording/animation, or (b) leaving snow unhandled/out of scope.
- **Scope check (asked, since it materially changed the size of this work):** the big rotating commentary box (`soupMessages.js`) has a lot of rain-specific imagery (umbrellas, puddles, storm drains) that reads oddly during snow. Asked whether that also needed a parallel snow-flavored set (~60 new lines) or should stay generic for now. **Answer: stay generic for now** — only `classifySoupcon`'s short title/reason text and the animation are type-aware; the commentary box is an accepted, documented gap, not an oversight.
- **Done:**
 - `src/lib/soupcon.js`: added `SNOW_KEYWORDS` (snow, sleet, blizzard, flurries, wintry mix), checked *before* `RAIN_KEYWORDS` so ambiguous phrases ("Snow Showers," "Rain and Snow") resolve to snow rather than being swallowed by a rain substring match. "Freezing Rain" stays rain (visually falls as rain, freezes on contact). Replaced the old boolean `periodIndicatesRain/observationIndicatesRain` with type-returning `periodPrecipType/observationPrecipType("rain"/"snow"/null)`; the numeric-probability fallback (no type info of its own) defaults to "rain" as the far more common case.

classifySoupcon's result now carries precipType (set for levels 1-3, null for 4/5) alongside type-specific title/reason text via a small PRECIP_WORDING lookup table — the numeric level and its ordering/priority logic are unchanged, only the type-detection and wording layer is new.

- src/lib/soupcon.test.js: renamed tests to match the new function names, added snow-specific coverage for all three levels, the ambiguous-phrase priority rule, the "Freezing Rain stays rain" rule, and a dedicated regression test reproducing the exact bug found above (bare "Snow," no probability, no longer falls through to level 5). 30/30 passing (up from 22).
- **Snow animation:** new src/components/SnowOverlay.jsx (a reintroduced version of FRTCON's original falling-snowflake effect — * glyphs with a horizontal-drift @keyframes snowDrift), alongside the existing RainOverlay.jsx (straight-falling rain streaks, unchanged). App.jsx picks whichever component renders based on soupcon?.precipType === "snow", defaulting to rain otherwise (levels 4/5 render zero of either way, per the existing density mapping, unchanged from before). The shared positional container class was generalized from .rain-overlay to .precip-overlay (pure layout, no type-specific styling) so both components reuse one CSS rule.
- **Docs:** README.md (scale explanation now covers the rain/snow split and the "commentary stays generic" caveat, project structure lists SnowOverlay.jsx, a new limitations bullet), CONTEXT.md (new architecture bullet on the rain-vs-snow design, and a new "Known gotchas" entry documenting the silent-misclassification bug as a general lesson about keyword-classifier fallback behavior, not just a changelog note).
- npm run lint, npm run test (30/30), and npm run build all pass.
- **Not verified live in a browser:** same recurring gap as most of this rebuild — the snow animation itself (does it read as snow, is the drift/density right) has been reviewed as code only, not seen rendered.

15. FRTCON.com cross-promo pill on snow — FIXED

- **Ask (per user):** "When snow is predicted, we should send them to FRTCON.com. Lets add a pill next to the facebook share. 'Snow soon, check your FRTCON!'"
- **Done:** a new <a> pill in App.jsx's .soupcon-status-row (next to the Facebook Share button, as asked), shown only when soupcon.precipType === "snow" (i.e. any of the three snow-triggering levels — currently snowing, snow in 12h, or snow in 48h — not just "currently snowing"). Links to https://frtccon.com with target="_blank" rel="noopener noreferrer", matching the safety pattern already used for the Facebook share's own window.open call. Styled as a pill (.snow-crosslink-pill in styles.css) reusing the app's existing blue accent color (#60a5fa/#1a0f2e, the same pairing .btn-primary already uses) rather than introducing a new color, and added to the same shared height/font-size selector group the badge/ alert-tag/share-button already share so it sits level with them on the row. The row's existing flex-wrap: wrap handles this longer pill gracefully on narrow widths (wraps to its own line rather than overflowing or squeezing its siblings).
 - README.md's feature list updated with a one-line mention.
 - npm run lint, npm run test (30/30, unchanged - this is presentation-only, no classification logic changed), and npm run build all pass.
 - **Not verified live in a browser:** same recurring caveat.
 - **Follow-up (per owner, after item 22):** frtccon.com is US-only, so the pill is now also gated on result.provider === PROVIDER_NWS and never shows for a non-US (Open-Meteo) result, even when it is snowing there. The snow overlay and wording are unaffected. Lint, tests and build pass; a JSX-only condition, not verified in a browser (no snow case exists to trigger it live).

16. Post-launch bug review — FIXED

- **Ask (per user):** "deep dive and analyze and make sure there are no bugs in the new implementation." Reviewed soupcon.js, weatherApi.js, cache.js, App.jsx, components, and sw.js, and probed live api.weather.gov (7 cities at once: hourly period timing, day/night sky wording, the 5 nearest stations' latest observations).

- **Done (clear-cut bugs, fixed):**
 - **Ended forecast periods counted as current.** Live NWS forecast/hourly responses led with an already-ended hour in 3 of 7 cities (e.g. Minneapolis at 14:29 CDT led with 13:00–14:00). That made the level-1 hourly fallback read a past hour as “right now” and cut the 12h/48h windows short by an hour. `classifySoupcon` now drops periods whose `endTime` has passed (hourly and extended), with an injectable now; 3 new tests (33/33 passing).
 - **Blank observations read as “not raining.”** Several stations, including Chicago’s *nearest* station (KMDW), posted fresh observations with an empty `textDescription` and no `precipitationLastHour`. `findFreshObservation` accepted them, which both read as “definitely dry” and, being non-null, suppressed the hourly fallback. It now also requires a non-blank description or a numeric precip reading. Verified live: Chicago now skips KMDW and uses KORD’s real reading.
 - `npm run lint`, `npm run test` (33/33), `npm run build` all pass.
- **Done (follow-up, owner-decided — “fix 3-6, include in-vicinity readings as local enough to count”):** these touch the locked-in keyword lists, so were raised as questions first rather than changed unilaterally.
 - **Partly Cloudy vs. Partly Sunny day/night asymmetry.** Live data (7 cities, all hourly periods) showed “Partly Sunny” only ever in daytime periods and “Partly Cloudy” only at night — the same sky cover, but “Partly Cloudy” matched cloudy (level 4) while “Partly Sunny” didn’t (level 5). Since the near-term window always includes nights, fair outlooks kept dropping to 4. Now `NOT_CLOUDY_PHRASES` strips “partly cloudy” before the cloudy check, so both are level 5; `Mostly Cloudy/Cloudy/Overcast/fog/etc.` are still level 4, and “Partly Cloudy then Mostly Cloudy” still reads as cloudy. The old test that locked in “Partly Cloudy → 4” was replaced.
 - **“Slight Chance” read as “It’s raining right now.”** The level-1 hourly fallback (no usable observation) now uses `currentPeriodPrecipType`: requires $\geq 40\%$ probability, or — when NWS reports none — precip wording without “chance.” Levels 2/3 are unchanged (any precip wording still counts there).
 - **Level 5 with rain in the extended forecast.** Precip in any of the next 4 extended periods now means level 4, closing the gap where rain past the 48 hourly periods, worded with no cloudy keyword (“Rain Showers,” seen live), fell through to “clear and sunny.”
 - **“In Vicinity” station readings:** kept counting as level 1, per owner (“local enough to count”) — no logic change; documented in a comment and locked in with a test.
 - 6 net new tests (39/39). Lint and build pass. Also ran the real classifier against live NWS data for 7 cities; all results were sensible (e.g. Chicago’s `Mostly Sunny/Partly Cloudy` outlook is now level 5, previously 4).
- **Done (second follow-up, per owner — “fix all four”):** the lower-severity findings from the same review.
 - **Sequential station lookups.** `findFreshObservation` walked up to 5 stations one at a time, each with its own 10s timeout, inside the lookup’s `Promise.all` — a slow NWS could hold “Looking up...” for ~a minute. Now fetched in parallel (worst case one timeout); the nearest *usable* station still wins, picked by station order rather than arrival order. Live timings across 7 cities: 145–1477 ms per full lookup.
 - **Aborted in-flight /points request reused.** `getGridpointInfo`’s de-dup map now stores each request’s signal and never joins one that’s already aborted (a new lookup starts its own); cleanup only deletes its own entry so it can’t evict a newer replacement.
 - **Empty hourly forecast → “Clear and sunny.”** `getHourlyForecast` now throws a user-facing “didn’t return a forecast” error (not cached) instead of returning []. Background refreshes that hit it keep the last good data, like any other refresh failure.
 - **“Ice Pellets”/“Hail” unrecognized.** Added ice pellets to `SNOW_KEYWORDS` (stations’ wording for sleet) and hail to `RAIN_KEYWORDS`.
 - New `src/lib/weatherApi.test.js` (5 tests, fetch stubbed); the 3 covering new behavior were confirmed to **fail against the pre-change `weatherApi.js`** and pass after, the other 2 guard behavior that shouldn’t change (de-dup still collapses concurrent live requests; nearest usable station wins). Plus a keyword test in `soupcon.test.js`. 45/45 passing; lint and build pass; live 7-city classifier run unchanged.
- **Not verified live in a browser** — logic changes only, verified via tests and live-API runs of the

classifier.

17. Observation station radius + snow pill wording — FIXED

- **Ask (per user), following item 16's review:** “why do we need 5 stations? Can we do a haversine and only include stations that are within 60 miles (but use a minimum of 2 stations regardless). Also fix the ‘snow soon’ wording.” Also confirmed: the 90-minute observation age limit is fine as-is.
- **Why 5 (answered):** an arbitrary cap from item 2, there because the nearest station is often stale or blank. Kept as an upper bound rather than dropped: with the stations fetched in parallel (item 16), an uncapped 60-mile radius would mean dozens of requests per lookup and per 5-minute refresh in dense areas (live counts within 60 miles: Seattle 41, Minneapolis 22, Chicago 14).
- **Done:**
 - New exported `haversineMiles` and `selectObservationStations` in `weatherApi.js`: sorts NWS's station list by computed distance (it isn't strictly sorted — confirmed live for Seattle, Minneapolis, Chicago), takes everything within 60 miles, at least the nearest 2 regardless of distance, at most 5. Stations with no coordinates sort last and can only fill the minimum.
 - Verified live: Seattle/Chicago → 5 stations within 24 miles; Ely, NV → its one in-range station plus one at 64 mi; Big Bend, TX → 26 mi + 72 mi; Jordan, MT → 5 within 53 mi, none with a usable reading, so it correctly fell back to the hourly forecast.
 - **Snow pill wording:** reads “Snow! Check FRTCON!” at level 1 (already snowing; wording chosen by owner); “Snow soon, check your FRTCON!” is kept for levels 2/3.
 - 6 new tests (selection rules + haversine sanity check); existing station tests given coordinates. 51/51 passing; lint and build pass.
 - **Not verified live in a browser** (the pill wording change is JSX only).
- **Browser verification (first real one this rebuild):** Claude in Chrome wasn't available, so a standalone headless Chrome (downloaded to scratch space, driven over the DevTools protocol, - -no-sandbox since this host's AppArmor blocks Chrome's sandbox; the snap Chromium won't start from a non-snap shell) was run against a vite preview of the production build. Confirmed working: page load + service worker registration; ZIP lookup (Seattle → SOUPCON 3, rain overlay, 4 commentary lines, alerts count); `soupcon_last_*` saved only on success and auto-resume on reload; invalid ZIP → friendly error without overwriting the saved ZIP; geolocation lookup (Boston → SOUPCON 1, 140 rain streaks, 3 real alerts rendered); recipe modal (focus moves in, scroll lock, Escape closes and returns focus to the menu button); Share (clipboard text matches on-screen box exactly, toast shown, Facebook tab opened); simulated snow via response rewriting (SOUPCON 1 → “Snow! Check FRTCON!” + snowflakes; SOUPCON 2 → “Snow soon, check your FRTCON!”); 390px phone width with no horizontal overflow. Only console error: the expected logged 404 for the invalid ZIP. **Still not verified:** the actual PWA install flow on a real device, and print preview.
 - **Recipe modal serif font (pre-existing from FRTCON) —  FIXED:** the modal renders outside `.app-page` (for print isolation), so it never inherited the app's Arial and fell back to the browser's default serif. `.modal-card` now sets the same font-family as `.app-page` (no effect on `IOSInstallHelp`, which already inherited it; h2 titles keep their own system-ui rule). Verified in headless Chrome: computed font is Arial on screen and under print media emulation; screenshot checked.
 - **Deliberately left as-is (per owner):** NWS alert descriptions keep NWS's own hard line breaks, so they wrap at about half the card width on desktop.
- **Deliberately left as-is (per owner, “leave that alone for now”):** an observation with a positive `precipitationLastHour` but no precipitation wording in `textDescription` still defaults to “rain” (`observationPrecipType`) — the reading has no type of its own. Also confirmed fine as-is: the 90-minute observation age limit.

18. Soup recipes from the owner's wiki — FIXED

Supersedes the “rotating soup recipes” deferral in item 4 with a concrete shape, agreed 2026-09-27 (waiting on the owner to send the wiki URL).

- **Source:** the owner's own wiki, where their wife types in her recipes. Licensing/attribution is a

non-issue (family-authored); no credit lines unless the owner asks for one.

- **Plan (proposed, owner hasn't objected):**

- Extract recipes from the wiki page once, into static app data (`src/data/recipe.js` becomes a list), not fetched live. No backend, wikis often block cross-origin reads, and a wiki edit can't break the app. New recipes = ask for a re-extract + redeploy.
- Reuse the existing `RecipeModal` (including its print view) for whichever recipe is chosen.
- Menu shape decided by count once the page is seen: a few recipes → one hamburger-menu item each; more than ~4-5 → a single "Soup Recipes" item opening a picker.
- Check the extraction against the page before wiring it in (wiki formatting varies); ask about anything ambiguous.

- **Open until the URL arrives:** whether the wiki is reachable from this machine (public, LAN-only, or login-gated — if gated, the owner exports the page instead). Live-loading from the wiki (auto-updating, but needs the wiki to allow cross-origin reads and makes the app depend on it) is the fallback if re-extract + redeploy gets tedious.

- **Done:**

- **Extraction:** the wiki (thoughtleap.com, the owner's own domain — see `CONTEXT.md`'s infra section) is reachable from this machine. Indexed exactly `Category:SOUPCON` via the MediaWiki API (`action=query&list=categorymembers`, no continue needed — only 2 pages currently exist in that category), then fetched each page's raw wikitext via `action=raw`. **WebFetch's AI-summarized output was caught and discarded before use** — it paraphrased ingredient/step text and even invented a display title ("Peanut Butter Chicken Soup") for the page actually titled "Senegalese Chicken Soup"; switched to a direct curl of the same `action=raw` URLs to get the exact wikitext instead. The `== Ingredients ==` `Instructions ==` sections were extracted, plus each page's personal narrative section (a "Soup Tales!" / cook's-notes writeup) where one exists — initially left out, then added back per the owner ("lets include that for flavor"); see the follow-up note below for the resulting shape.
- **Static data, per the proposed shape:** `src/data/recipe.js` (the single hardcoded placeholder) replaced with **one file per recipe** under `src/data/recipes/` (`potstickerSoup.js`, `senegaleseChickenSoup.js`), each a plain `{ title, ingredients, steps }` object — same shape the old single `RECIPE` had, not a new array/list data file, since the owner specifically asked for each recipe in its own file, imported separately, rather than a data-driven loop.
- **Menu shape:** 2 recipes → one hamburger-menu item each, per this item's own "a few recipes" branch (no picker needed). `App.jsx`'s single `recipeOpen` boolean replaced with `openRecipe` (holds the selected recipe object, or null); two dropdown items ("Potsticker Soup", "Senegalese Chicken Soup") each set it directly.
- **RecipeModal** now takes a recipe prop instead of importing a hardcoded `RECIPE`, so the existing modal (including its print view) is reused for whichever recipe is open, unchanged otherwise.
- **Verified live**, not just via tests: `npm run lint`, `npm run test` (51/51 — unchanged, this item touches no classification logic), `npm run build` all pass. Also drove a vite preview build with the same scratch-space headless Chrome / raw DevTools-protocol approach item 17 used (no puppeteer-core installed, so scripted directly over a native Node WebSocket) and confirmed live: both menu items appear with the right labels, each opens the correct recipe's modal (title, 14 Potsticker ingredients, 11 Senegalese steps all matched the source page).
- **Not done here, flagged by the owner as a future idea, not this item's scope:** recipes currently still render as an overlay modal (reusing `RecipeModal` as-is); dedicated per-recipe pages are a possible later change, not built now — added to `CONTEXT.md`'s shelved-work list.

- **Done (follow-up, per owner — "lets include that for flavor"):** the "Soup Tales!" narrative section, initially left out, is now part of the recipe data and rendered.

- Recipe shape gained two optional fields, `storyHeading/ storyParagraphs` (only present when the wiki page actually has a narrative section — Senegalese Chicken Soup currently doesn't have one, so its file omits them). `RecipeModal.jsx` renders the section conditionally (`recipe.storyHeading && recipe.storyParagraphs`) below Steps, reusing the existing `.modal-paragraph` class (already used by `IOSInstallHelp.jsx`) rather than adding new

- CSS — no `styles.css` changes needed.
- **Re-fetched both pages fresh rather than reusing the earlier extraction** (per the “we’ll be indexing this regularly” heads-up), and the Potsticker page had already been revised since the initial extraction — ingredient list updated to match (1 tbs not 1 TBSP, ham or char siu, ginger amount/jarred-alternative reworded, chopped parsley in place of chopped cilantro).
 - **Two content quirks found in the live source, flagged rather than silently resolved:** (1) a likely typo, “sliced finr” — corrected to “sliced fine” as an obvious spelling fix; (2) the ingredients list now says “chopped parsley” but the instructions still say “Stir in the cilantro and serve” — a real inconsistency in the source content (probably a partial edit), transcribed verbatim rather than guessed at, since fixing it belongs upstream on the wiki page and would otherwise just get overwritten by the next re-index anyway.
 - `npm run lint`, `npm run test` (51/51, unchanged), `npm run build` all pass. Verified live in headless Chrome: the “Soup Tales! Woo! Ooo!” heading and all 8 paragraphs render for Potsticker Soup; Senegalese Chicken Soup’s modal correctly shows zero story paragraphs (no section rendered) since that page has none.
 - **Memory updated** (`recipe-source-own-wiki.md` in the assistant’s persistent memory, not this repo) with the extraction procedure, including the `curl-not-WebFetch` gotcha and how to handle repeat indexing runs (match by page title, update in place vs. new file, flag rather than auto-delete a page that drops out of the category).
 - **Done (re-index, 2026-09-28):** owner asked to re-ingest the Potsticker recipe after a wiki edit. The wiki page itself was renamed to Potsticker (Dumpling soup) (still the sole `Category:SOUPCON` member matching the old Potsticker file, matched by content not filename per the memory’s matching rule) — the app’s displayed recipe title and hamburger-menu label were **left as “Potsticker Soup”**, since that’s an app-level display choice, not wikitext content, and nothing asked for it to track the wiki’s internal page title; flagging here in case the owner wants it changed.
 - **Recipe content substantially revised upstream:** the dumpling dipping sauce is now made from scratch as its own step (soy sauce, rice vinegar, mirin, honey, sesame oil, granulated garlic, granulated onion) rather than the old “20 frozen potstickers with their sauce” assumption; $\frac{1}{3}$ cup dry sherry dropped; a Garnish group (parsley, sesame seeds, green onions) added; steps re-ordered/reworded to match. Story section also reworded/expanded (still 8 paragraphs) — transcribed verbatim per the no-paraphrase rule.
 - **Schema question flagged, not resolved:** the source page now groups ingredients under sub-headings (main list / Potsticker Sauce / Garnish) via wiki `===` headers, but the recipe data shape (`{ title, ingredients: string[], steps, storyHeading?, storyParagraphs? }`) has no concept of ingredient groups, and `RecipeModal.jsx` renders ingredients as one flat `<ul>`. Rather than changing the shape/component (out of scope for a data re-ingest), “Potsticker Sauce:” and “Garnish:” were kept as plain string entries inline in the flat ingredients array, in source order — they render as ordinary list items, not as real sub-headers. If more recipes bring grouped ingredients, worth a real fix (e.g. a groups shape).
 - **Real content inconsistency in the source, transcribed as-is per the verbatim rule (not silently fixed):** step 6 says “Add the broth, potsticker sauce, **mirin**, ham, ginger...” even though mirin is already mixed into the potsticker sauce in step 1 — looks like a leftover from the previous sherry ingredient’s wording that wasn’t fully updated when the recipe was reworked. Belongs upstream.
 - Also picked up the still-present “Makes 4 servings.” line at the top of the page — **not added to the data**, since the recipe shape has no servings field and `RecipeModal.jsx` doesn’t render one; flagging as a possible future field rather than inventing a place to put it.
 - `npm run lint`, `npm run test` (57/57 — grown from the 51/51 recorded above from other work since, unrelated to this change), `npm run build` all pass.
 - **Done (re-index, 2026-09-28, new recipe):** owner asked to ingest Lasagna Soup from the wiki. Fetched via `action=raw` (an initial `WebFetch` pass again paraphrased the steps — discarded, per the memory’s warning). New `src/data/recipes/lasagnaSoup.js`, imported and wired into `App.jsx` (menu item + `ALL_RECIPES`), 3 recipes so still one menu item each. Its Herb Mix/Cheese Topping ingredient groups use the inline “Name:” convention, which `RecipeModal` now renders

bold. No servings line, no narrative section on the page, so neither is included. Category: SOUPCON also contains Potato Leek Soup, **not ingested** (only Lasagna was requested) — flagging it.

19. “Soup of the day” pill — **FIXED**

- **Ask (per user):** “lets pick a soup of the day and create a pill button just to the right of Facebook share. Cache the chosen soup until midnight local time and pick a new one at random the next day the site is accessed. Name it ‘Lets make {soupname}!’”
- **Done:**
 - **src/lib/soupOfTheDay.js** — new pickSoupOfTheDay(recipes, { now, random }), injectable now/random for tests (same pattern classifySoupcon already uses for now). Caches { date, title } in localStorage under soupcon_soup_of_the_day, keyed off a local-calendar-date string (getFullYear/getMonth/getDate, not UTC) rather than a rolling TTL like cache.js’s other entries — this needed to flip specifically at local midnight, however long that is from the last pick, which a duration-based TTL can’t express. Not folded into cache.js’s getItem/setItem for the same reason. If the cached title no longer matches any current recipe (one was removed), or localStorage throws (private browsing, blocked site data), it falls back to picking fresh rather than erroring.
 - **App.jsx:** a module-level ALL_RECIPES array ([POTSTICKER_SOUP, SENEGALESE_CHICKEN_SOUP]) feeds the picker — the per-file/ separate-import pattern item 18 set up for the menu is unchanged; this is a second, small, explicit use of the same imported objects, not a rotation mechanism replacing the menu. A new recipe needs adding to this array too, alongside its menu item and import (flagged in an inline comment). soupOfTheDay is computed once via a lazy useState initializer (so there’s no flash of a missing pill on first render, unlike a useEffect-after-mount approach) and clicking the pill opens that recipe’s modal via the existing openRecipe state — same mechanism the menu items use.
 - **Placement:** rendered directly after the Share button and before the snow crosslink pill in .soupcon-status-row, per “just to the right of Facebook share.”
 - **Label:** exactly Lets make {title}! (no apostrophe), per the user’s own wording, not autocorrected to “Let’s.”
 - **Styling:** .soup-of-the-day-pill in styles.css — same pill shape/sizing as .snow-crosslink-pill (added to the shared height/font-size selector group), but the condition-status box’s amber (#f59e0b/#3f2b00) instead of the blue accent, so it reads as its own warm/food-coded thing next to the blue Share button and snow pill rather than blending in.
 - **Tests:** new src/lib/soupOfTheDay.test.js (6 tests) — empty/null recipe list, injected-random selection, same-day caching (a second call with a different random draw and a later same-day timestamp still returns the first pick, and the second random function is never even invoked), a fresh pick once local midnight has passed, a stale cached title (recipe removed) falling back to a fresh pick, and a thrown localStorage access still returning a pick. Stubs a working in-memory localStorage for these tests specifically (real localStorage isn’t available under vitest’s default node environment — see weatherApi.test.js’s note — so this is the first test file in the project to actually exercise real caching behavior rather than treating every cache read as a miss).
 - npm run lint, npm run test (57/57, up from 51), npm run build all pass. **Verified live** in headless Chrome against a vite preview build with a real ZIP lookup (Seattle): pill renders in the right position with the exact label text, clicking it opens the matching recipe’s modal, and the pick is written to localStorage and still matches after a full page reload (same local day). Debugging note for next time: setting <input>.value directly and dispatching a plain input event does **not** update a React controlled input in this app — needs the native HTMLInputElement.prototype.value setter called via .call() first, then the input event, or React’s own change handler never fires (confirmed by comparing a script that hung with one that didn’t).
 - **Not tested:** the actual local-midnight rollover in a real browser session (would require leaving a tab open across midnight, or manipulating system clock) — covered instead by the injectable-now unit tests above, same tradeoff classifySoupcon’s own “ended forecast periods” tests made.
- **Follow-up (per owner, 2026-09-30, found in item 25):** “soup of the day should expire

at midnight regardless of whether the app is left open.” It was picked only on mount, so a tab or installed PWA left open kept yesterday’s soup. App.jsx now re-picks on a timer set for the next local midnight (+1 s so the date has flipped), re-arming itself each day, and also re-checks whenever the page becomes visible again, since background tabs and suspended PWAs can freeze the timer. Re-checking mid-day is harmless: pickSoupOfTheDay returns the same cached recipe object for the same date, so nothing changes until the date does. New exported msUntilLocalMidnight(now) in soupOfTheDay.js builds the next midnight from the calendar date (not now + 24 h, so a DST change can’t shift it an hour); 2 new tests (145/145). Lint and build pass. **Not verified in a browser** (the rollover itself would need a clock change or an overnight wait; the timing math is unit-tested).

20. “Sources” panel (was ?stations=1 debug panel) — **FIXED**

- **Trigger (per user):** ZIP 12577 showed SOUPCON1 with nothing on radar and no rain. Investigated live: the nearest station (KSWF) was stale/blank so was skipped, and the next (KMGJ, 8.5 mi) had a single “Light Rain” observation at 19:20Z with no raw METAR, sandwiched between “Cloudy” readings (its 19:17Z METAR showed no precipitation), while the hourly forecast said 2%. So one glitchy reading from the nearest usable station won level 1. Asked how to guard against that; owner chose instead to add a way to see the data behind the score: “if the user adds stations=1 or stations=true then, under the alerts, show the other results from the NWS that are driving the condition status.”
- **Done:**
 - weatherApi.js: new getStationReadings (debug only, uncached, one request per candidate station) returns each candidate’s id, distance, timestamp, description, precip reading, usable and chosen flags, using the same station selection and usability rules as getCurrentConditions. selectObservationStations now wraps a new selectObservationStationsWithDistance (same behavior).
 - New StationDebugPanel.jsx, rendered under the alerts in App.jsx only when the URL has stations=1/stations=true (parsed once at module load). Shows station readings (USED/skipped), the next 12 hourly and 4 extended periods with the precip type the classifier reads from each. Refetches whenever the main data refreshes. Small .station-debug-* styles.
 - 1 new test (58/58); lint and build pass.
 - **Follow-up (per owner):** heading changed from “Condition data (debug)” to a collapsed “Sources +/-” toggle; when expanded, the three data types render as one card with a header per type.
 - **Follow-up (per owner):** the stations URL parameter was removed; the Sources panel is always shown (collapsed). Station readings are now fetched only while it’s expanded, so normal lookups make no extra requests.
 - **Deliberately not changed:** the classifier itself still trusts the nearest usable station’s latest reading, so a single bad observation can still produce SOUPCON1. Options raised (not chosen): require forecast corroboration, require a second station.
 - **Not verified in a browser** (code, tests and live-API checks only).

21. lat/Lon URL parameters — **FIXED**

- **Ask (per user):** “add lat and lon parameters to customize the location being loaded” (URL query parameters).
- **Done:** App.jsx reads ?lat=..&lon=.. once at module load (readUrlLocation); both must be present, numeric, and in range (-90..90 / -180..180), else ignored. When valid, the mount effect runs runLookupFromCoordinates with source “url” instead of the saved-source auto-resume. Deliberately **not** persisted to soupcon_last_source/ soupcon_last_zip, so a one-off shared link doesn’t replace a visitor’s own remembered method. Coordinates outside NWS coverage surface NWS’s normal lookup error.
 - **Follow-up (per owner):** while the URL location is what’s being used (source === “url”, i.e. until the visitor runs a ZIP/browser lookup), “Using Lat: {lat} Lon: {lon}” shows to the

- right of the Search ZIP button (.custom-location-note), so it's visible the parameters were accepted.
- README.md updated. Not verified in a browser (lint/tests/build only).

22. Worldwide weather via Open-Meteo (non-US locations) — FIXED

Raised as a question after item 21 ("is there another world weather service we can query for weather outside the US"). Written as a plan first, then built step by step (each step's **Done:** note below). Before this item, NWS (api.weather.gov) covered US points only, so a lat/lon outside the US just surfaced NWS's lookup error.

- **Decision (proposed):** keep NWS for US coordinates (unchanged behavior, including alerts and station observations) and fall back to **Open-Meteo** (api.open-meteo.com) for everything else. Chosen because it needs no API key or backend (matches the "no secrets, fully static" constraint), has CORS enabled, and covers the world. Alternatives considered and not chosen: MET Norway (requires an identifying User-Agent, the same browser problem as NWS — see CONTEXT.md), and OpenWeatherMap/WeatherAPI/Tomorrow.io/Visual Crossing (all need an API key, which would be exposed client-side without a proxy).
- **Terms check before shipping:** Open-Meteo's free tier is for non-commercial use. Fine for the app today; revisit if ads or the affiliate ideas (see "Shelved" in CONTEXT.md) are ever added.

Steps:

1. **Button/label copy:** change the "Search ZIP" button to "**Search US ZIP**" (App.jsx, including its "Looking up..." sibling state as needed), and make the ZIP input's placeholder/aria-label say US ZIP too (currently "Enter ZIP code"/"ZIP code"). isValidZip and zippopotam.us lookups stay US-only, so the wording should say so; non-US users get in via browser location or the lat/lon URL parameters. Update README.md and public/index.md wording where they say "ZIP code". **Done:** the button now reads "Search US ZIP", the input's placeholder/aria-label/hidden label say "US ZIP code", and the three geolocation/ZIP-fallback error messages in App.jsx say "US ZIP code" too. README.md and public/index.md updated. The "Looking up..." busy state is unchanged (it didn't mention ZIP). Lint, tests and build pass; not verified in a browser.
2. **New API layer** (e.g. src/lib/openMeteoApi.js, or alongside weatherApi.js): fetch hourly (precipitation_probability, precipitation, weather_code, cloud_cover), daily, and current for a lat/lon in one request, using the same { signal, skipCache } options and cache.js TTL/prefix pattern as the NWS functions (new soupcon_openmeteo_* prefixes, registered in sweepExpiredCache's TTL map). Verify field names against live responses rather than assuming, as items 2 and 16 did for NWS. **Live probe (2026-09-30, before building):** (a) CORS is open (access-control-allow-origin: *), no key. (b) One request works for all three inputs: https://api.open-meteo.com/v1/forecast?latitude=..&longitude=..¤t=weather_code,precipitation,precipitation_probability (c) Use timeformat=unixtime: with timezone=auto the times are local strings with no offset (2026-09-30T17:30), which is easy to misread; unixtime gives the period start, so endTime = start + 3600 s for hourly (86400 s for daily). (d) By default hourly starts at local midnight today (168 rows for 7 days), i.e. it includes already-ended hours; forecast_hours=48&past_hours=0 starts at the current hour (verified: first row at or just before now), so it matches what NWS gives us. (e) current is a 15-minute model value (interval: 900) with weather_code, precipitation, rain, showers, snowfall. (f) daily gives weather_code, precipitation_probability_max, precipitation_sum, one row per calendar day (not 12-hour periods like NWS /forecast), so the level 4/5 "next four periods" window needs redefining (e.g. the next 2 days). (g) WMO codes actually seen across 11 cities: 0, 1, 2, 3, 45, 51, 53, 55, 61, 63, 80, 81, 95. No snow codes (71-77, 85-86) or freezing-rain codes (56, 57, 66, 67) turned up in late September, so the snow mapping is unverified against live data; recheck in winter or against a cold-climate city. (h) Errors: an invalid coordinate returns HTTP 400 with {"error":true,"reason":"Latitude must be in range of -90 to 90°."}. Given: 999.0."}. **NWS non-US probe (feeds step 6):** api.weather.gov/points/51.5074,-0.1278 (London) and Vancouver both return HTTP 404 with application/problem+json, type: "https://api.weather.gov/problems/InvalidPoint", title "Data Unavailable For Requested Point". Puerto Rico, Hawaii, Alaska and Guam all return 200, so a bounding box would have been wrong for territories and NWS-first routing is confirmed as the

right choice. Routing should key on that 404 (ideally the InvalidPoint type), not on any error. **Done:** new `src/lib/openMeteoApi.js` with `getOpenMeteoForecast(lat, lon, { signal, skipCache })` and `makeOpenMeteoUrl`. It makes the single request from the probe (current + hourly + daily, `forecast_hours=48&past_hours=0, timeformat=unixtime&timezone=UTC`) through the existing `fetchJson`, and returns `{ current, hourly, daily }` exactly as Open-Meteo shapes them (parallel arrays keyed by field). Nothing is normalized yet; that is step 3. Errors reuse `friendlyMessage` (now exported from `weatherApi.js`, no behavior change), so an HTTP failure reads as the same plain-language message as an NWS one. A response with no hourly/daily rows or no current block throws and is not cached, the same “empty forecast is an error, not clear” rule as NWS. `cache.js` gained `OPEN_METEO_CACHE_PREFIX` (`soupcon_openmeteo_`), a 15-minute TTL (matches Open-Meteo’s 15-minute current interval), `makeOpenMeteoCacheKey`, and an entry in `sweepExpiredCache`’s TTL map. New `src/lib/openMeteoApi.test.js` (4 tests: URL parameters, a successful single-request fetch, the empty-hourly failure, and the friendly HTTP-error message); 62/62 tests, lint and build pass. Also re-ran the exact query live for Tokyo with the app’s Accept header: 48 hourly rows, 7 daily rows, and the field names match. **Not wired into the app** (`App.jsx` and `weatherApi.js` still only use NWS), so there is no user-visible change yet.

3. **Normalize to the classifier’s inputs:** the goal is to keep `classifySoupcon`’s shape (hourlyPeriods, observation, extendedPeriods) rather than fork the scale logic. Map Open-Meteo output into that shape with a WMO-code lookup table: rain (51–67, 80–82, 95–99), snow (71–77, 85–86), fog (45, 48), overcast (3), plus `precipitation_probability`. Prefer codes over keyword-matching text, but either feed synthesized `shortForecast` strings the existing keyword lists already recognize, or add a code-based path — decide when built and record why. Synthesized periods need `startTime/endTime` so the ended-period filter from item 16 still works. **Done (per owner: a separate code path, plus human-readable text for the Sources block):** three new modules, none wired into the app yet. `src/lib/wmoCodes.js`: one table of every WMO code Open-Meteo can return (0–3, 45, 48, 51–57, 61–67, 71–77, 80–82, 85–86, 95–99) giving a readable label (“Slight rain”, “Overcast”), `precipType`, and a cloudy flag; `describeWmoCode` returns an unknown code as “Weather code N” rather than guessing. The same table drives the classifier and (step 8) the Sources panel, so the on-screen wording cannot drift from what decides the score. Freezing rain/drizzle and thunderstorms with hail are rain (the same calls the NWS keyword lists make); overcast, fog and rime fog are cloudy; partly cloudy is not. `src/lib/soupconOpenMeteo.js`: `normalizeOpenMeteo` flattens the raw parallel arrays into rows (start/end in ms, label, precip type, cloud cover, probability, amounts; the current block; daily rows for the Sources display), and `classifyOpenMeteo({ forecast, now })` applies the same ordered 1–5 checks as `classifySoupcon` and returns the identical result shape. **Refactor in `soupcon.js`:** a new exported `soupconResult(level, precipType)` builds the result object (title/reason wording included) and `classifySoupcon` now uses it, so both providers share one copy of the wording; behavior unchanged. **Level 4 vs. 5 as agreed:** the hourly rows (ended hours dropped) are split into four 12-hour blocks (the same ~48 hours as NWS’s four 12-hour periods); a block is cloudy if any hour has a cloudy code or its average `cloud_cover` is at least `CLOUDY_BLOCK_MIN_COVER` (70, an estimate of where NWS starts saying “Mostly Cloudy”, not measured against NWS output). The daily rows are not used for the score. **Deviation from what was proposed, flagged for the owner:** the hourly rows decide rain/snow by WMO code only; the probability is display-only. Reason: the NWS 40% probability rule was a stand-in for missing type text, and Open-Meteo’s codes already encode whether precipitation is expected, whereas an 80% probability under an “Overcast” code (seen in Reykjavik) would otherwise call rain. Easy to add a bar if you would rather have one. **Tests:** `wmoCodes.test.js` (6 groups) and `soupconOpenMeteo.test.js` (normalization, every level for rain and snow, the 12h/48h boundaries, the 69/70 cloud-cover boundary, per-block averaging, ended hours, probability ignored, identical result shape to `classifySoupcon`); 100/100 tests, lint and build pass. **Checked live** by classifying real responses for London, Mumbai, Reykjavik, Manaus, Oslo, Sydney and Cairo: results were plausible (e.g. Reykjavik level 2 with drizzle in the next hours; Cairo level 4 because later blocks average 85–90% cover with overcast hours). No snow case exists live in late September, so the snow paths are covered by tests only.
4. **“Currently raining” source:** Open-Meteo’s current block is model-based, not a station reading, so the NWS staleness/blank-station logic doesn’t apply. Decide whether it feeds level 1 directly

or only via the hourly fallback, and keep the 40%-probability bar from item 16 consistent for the fallback. **Done in step 3** (the classifier cannot work without a level 1 rule): current feeds level 1 directly through currentPrecipType: measured snowfall above 0 is snow (snow wins, as in the NWS text rules), then rain or showers above 0 is rain, then the current WMO code, then a bare precipitation above 0 as rain. There is no hourly-forecast fallback and no 40% bar, because current is always present in a valid response (a response without it is rejected in step 2). current is a model value, not a station reading, so a single bad observation cannot produce SOUPCON1 the way it can with NWS stations.

5. **Location label:** Open-Meteo's geocoding API only searches by name, so getLocationLabel has no direct equivalent. Options: a free client-side reverse geocoder (e.g. BigDataCloud), or fall back to showing the coordinates. Pick one, and handle its failure without failing the whole lookup (label is cosmetic). **Done, with a constraint found by probing that changes the plan:** new src/lib/reverseGeocode.js with getReverseGeocodedLabel(lat, lon, { signal }), the pure formatPlaceLabel(place), and coordinatesLabel(lat, lon). Provider: BigDataCloud's free client-side endpoint (api.bigdatacloud.net/data/reverse-geocode-client), no key, CORS open, checked live across 10 places. **Its Fair Use Policy only permits calls that use the device's own current location (browser geolocation), not pre-stored or third-party coordinates, and a breach can get the visitor's IP banned (HTTP 402).** So the geocoder must only be called for a browser-geolocation lookup, never for ?lat=&lon= URL coordinates; those get coordinatesLabel (e.g. "Lat 51.51, Lon -0.13", so a negative coordinate is not mistaken for a separator) instead. (Non-US ZIP lookup is out of scope, so ZIP never reaches this path.) Alternatives considered and not chosen: Nominatim (OSM; public server asks for low volume and attribution, and is not built for per-visitor lookups), and a paid/keyed geocoder (needs a proxy). **Label format:** "City, Country" using Intl.DisplayNames for the short country name ("London, United Kingdom", not BigDataCloud's "United Kingdom of Great Britain and Northern Ireland"); when the returned "city" is really an administrative area (real case: "Metro Vancouver Regional District") the more specific locality is used ("Downtown Vancouver, Canada"); open water gives just its locality ("Atlantic Ocean"); nothing named gives null so the caller uses coordinates. **Failure handling:** a geocoder failure (including a 402 ban) returns null rather than throwing and is not cached, so it can never fail the forecast lookup; an aborted lookup still throws like every other fetch. Successful labels are cached 1 hour (soupcon_geocode_prefix, makeGeocodeCacheKey, in sweepExpiredCache's TTL map). **Known cost:** fetchJson's 10 s timeout applies, so a hanging geocoder could hold the lookup up to 10 s if step 6 awaits it in the same Promise.all as the forecast; step 6 should not let the label delay the result (e.g. show the coordinates first and swap the name in, or use a shorter timeout). reverseGeocode.test.js (10 tests, using trimmed real responses): formats, the admin-area rule, ocean, null, coordinates, request URL, failure to null, abort still throws. 110/110 tests, lint and build pass. Not wired into the app yet.
6. **Provider routing** (needs steps 2-5 first, since there's nothing to fall back to until the Open-Meteo layer exists): decide US vs. non-US per lookup. Simplest: try NWS /points first and fall back to Open-Meteo on its "outside coverage" error (a 404 for non-US points), rather than maintaining a bounding box (which would misjudge borders, Puerto Rico, Alaska/Hawaii, etc.). Check the exact NWS error shape against live api.weather.gov for a non-US point early (a read-only probe, can be done any time before this step), and make sure the error is distinguishable from a genuine NWS outage (an outage must still fail, not silently switch provider). **Constraint from step 5:** the reverse geocoder may only be called when the lookup came from browser geolocation; for ?lat=&lon= URL coordinates use coordinatesLabel and never call it. **Done:** new src/lib/weatherProvider.js and the wiring in App.jsx. lookupWeather(lat, lon, { signal, skipCache, source }) starts the NWS alerts request, then awaits getLocationLabel (the first NWS call, which resolves the shared gridpoint); if that throws the new OutsideNwsCoverageError it returns an Open-Meteo result, otherwise it finishes the NWS lookup as before. **weatherApi.js:** getGridpointInfo now throws OutsideNwsCoverageError (exported, same user-facing message as before) for an HTTP 404 from /points, and only that; any other failure, including a 500 or a timeout, still fails the lookup, so an NWS outage never silently switches provider. It keys on the 404 status, not the InvalidPoint problem type, because fetchJson does not read error bodies and /points only 404s for this case. The NWS alerts request is started up front but its failure is ignored until NWS is confirmed to cover the point (a non-US point makes it 4xx too and that must not beat the coverage error).

Result shapes: { provider: "nws", locationLabel, hourlyPeriods, extendedPeriods, observation, alerts } or { provider: "open-meteo", locationLabel, openMeteo, alerts: [] }; App.jsx spreads it into result next to source/lat/lon/fetchedAt, and classifyLookup(result) picks classifySoupcon or classifyOpenMeteo. **Place name:** for Open-Meteo, only source === "browser" calls the reverse geocoder; "url" (and anything else) shows coordinatesLabel. The geocoder wait is capped at GEOCODE_WAIT_MS (3 s): after that the coordinates are shown so a slow geocoder cannot hold up the forecast, and the geocoder request keeps going and caches the name for the next lookup of that place. **Refresh (step 9 in this plan, done here because the routing needs it):** refreshWeather(lat, lon, provider) refreshes from the provider the lookup used and does not re-probe NWS; refreshWeatherData in App.jsx now takes the provider and its two effects depend on result.provider. **Tests:** weatherProvider.test.js (10 tests, fetch stubbed): US point uses NWS and never touches Open-Meteo or the geocoder; a 404 falls back; a 500 does not fall back; browser source uses the geocoder; URL source shows coordinates and never calls it; a failed geocoder and a hanging one both give coordinates; refresh for each provider; classifyLookup for each. 120/120 tests, lint and build pass. **Checked live** (real APIs, run from node with the URL source): Seattle and San Juan, PR route to NWS ("Seattle, WA", "San Juan, PR"); London, Vancouver and Mumbai route to Open-Meteo with coordinate labels and plausible levels. The browser-geolocation geocoder path was deliberately not run live (the policy forbids other-than-device coordinates; it is covered by the stubbed tests). **Known intermediate rough edges, fixed by the next steps, not oversights:** an Open-Meteo result still renders the "0 alerts" tag and the "No active alerts" card (step 7), and opening the Sources panel for one shows a station-load error because that panel only knows NWS data (step 8). **Not verified in a browser.**

7. **Alerts panel:** Open-Meteo has none. For non-US lookups, hide the alerts count tag and the "Active Alerts" section (or say alerts aren't available outside the US) rather than showing a misleading "0 alerts." **Done:** in App.jsx, the alert-count tag is shown only when result.provider === PROVIDER_NWS, and for an Open-Meteo result the "Active Alerts" section shows one card reading "Weather alerts are only available for US locations." instead of "No active alerts were returned" (which would wrongly imply there are none). The NWS display is unchanged. Lint, 120/120 tests and build pass; a JSX-only change with no new unit test (nothing in the project renders components under test). **Not verified in a browser. Revised (per owner):** the explanatory "Weather alerts are only available for US locations." card is gone too; the whole "Active Alerts" section (heading and card) is now hidden for an Open-Meteo result, along with the count tag, so a non-US result shows no alerts UI at all. The item 7 and item 11 notes above that describe the explanatory card are superseded by this.
8. **Sources panel (StationDebugPanel):** it shows NWS stations and forecast periods. Make it provider-aware: for Open-Meteo, show the hourly/daily data and mark station readings as not applicable. **Done:** StationDebugPanel.jsx is provider-aware. For an Open-Meteo result it no longer fetches stations (the effect is skipped) and instead renders a new exported OpenMeteoSources: (1) "Current conditions (model estimate)" with the label and precip type that decide level 1, plus rain/showers/snowfall amounts, cloud cover and time; (2) the next 12 hourly rows with the readable weather label, cloud cover, precip chance and [rain]/[snow]; (3) "Cloud cover by 12-hour block (next 48 hours)", each block with its average cover and whether it is cloudy and why ("overcast or fog hour" vs. "high average cover"), so a level 4 vs. 5 result can be traced; (4) the daily outlook (label, max precip chance, precipitation), marked display-only. NWS lookups render exactly as before. **Supporting changes:** soupconOpenMeteo.js now exports cloudBlocks(hourly) and upcomingRows, and classifyOpenMeteo uses cloudBlocks so the panel shows the same blocks the classifier used (behavior unchanged, covered by the existing tests). **Fixed while here, found by probing for this step:** the Open-Meteo request used timezone=UTC, which makes the daily rows UTC days, misaligned with the location's own days (e.g. Tokyo). It is now timezone=auto with timeformat=unixtime: checked live for Tokyo that hourly rows are still true instants (first row is the current hour) and daily rows start at the location's local midnight. The response's IANA timezone is now kept (getOpenMeteoForecast returns it, normalizeOpenMeteo carries it) so the panel labels each day in the place's own zone; hourly times still show in the viewer's zone, like the NWS panel. The step 2 note above that says timezone=UTC describes the original request; this supersedes it. **Tests:** cloudBlocks and timezone cases added to soupconOpenMeteo.test.js, and a new StationDebugPanel.test.jsx renders OpenMeteoSources to static HTML (no DOM test

setup exists in this project) and checks the readable text and the absence of “undefined”/“NaN”; it needed esbuild: { jsx: "automatic" } in vite.config.js for vitest’s JSX transform, applied only when mode === "test" because the real build uses oxc and warns if both are set. 126/126 tests, lint and build pass. **Not verified in a real browser** (the static render covers the content, not the expand/collapse interaction, which is unchanged).

9. **Refresh path:** refreshWeatherData and runLookupFromCoordinates in App.jsx call the NWS functions directly; route them through the same provider choice (and remember which provider a result came from so a background refresh doesn’t re-probe NWS every time). **Done in step 6** (see there): refreshWeather and the provider-aware refreshWeatherData.
10. **Tests:** vitest coverage for the WMO-code mapping (each of rain/snow/ fog/overcast/clear at each level), the provider fallback decision (non-US error falls back; other errors don’t), and normalization output feeding classifySoupcon to each level, with fetch stubbed as in weatherApi.test.js. **Done (written alongside steps 2-8 rather than as a separate pass, checked against this list now):** every item here has tests, 126 in the project in total. WMO mapping: wmoCodes.test.js (labels, every rain code, every snow code, freezing rain as rain, cloudy vs. not, unknown code). Fallback decision: weatherProvider.test.js (a US point uses NWS and never touches Open-Meteo or the geocoder; a 404 falls back; a 500 does not; browser vs. URL source for the place name; a failed and a hanging geocoder both give coordinates; refresh per provider; classifyLookup per provider). Normalization feeding the classifier to each level: soupconOpenMeteo.test.js (levels 1 to 5 for rain, snow at levels 1 and 3, freezing rain as rain, the 12h/48h boundary, the 69/70 cloud-cover boundary, per-block averaging, ended hours ignored, probability ignored, result identical to classifySoupcon’s for the same level). Also covered: openMeteoApi.test.js, reverseGeocode.test.js and the Sources panel render. fetch is stubbed throughout, as in weatherApi.test.js. **Not covered by unit tests:** the URL-parameter parsing in App.jsx (inline, module-load) and the components’ interactive behavior; both need a browser (step 11).
11. **Docs and verification:** update README.md (APIs used, what it does, known limitations: no alerts outside the US, model-based current conditions) and CONTEXT.md (provider routing decision, why Open-Meteo, terms caveat). Verify live against a few non-US cities in a real browser via the lat/lon URL parameters (see CONTEXT.md’s browser-testing note), including one rain and one clear case. **Done:** README.md (what it does, a new “Worldwide lookups” section, APIs used, known limitations, file list), CONTEXT.md (a new architecture bullet with the routing decision, the classifier design calls, the request details, the BigDataCloud Fair Use constraint, the Open-Meteo terms caveat and what is out of scope; the browser-testing note extended) and public/index.md updated. **Verified in headless Chrome** (chrome-headless-shell driven with puppeteer-core against vite preview of the production build, real NWS/Open-Meteo APIs): Seattle via ?lat=&lon= gives “Seattle, WA” from NWS with the alert count and “No active alerts”; London and Mumbai via URL route to Open-Meteo with headlines like “#SOUPCON4 CONDITION: Lat 51.51, Lon -0.13 is currently at Soup Condition #4.” and “Weather alerts are only available for US locations.”; the “Using Lat: Lon:” note shows beside “Search US ZIP”; an out-of-range lat or a lone lat is ignored; the Sources panel for London shows the model current conditions, hourly rows with readable labels, the four cloud blocks with reasons and the daily outlook, and the page had no JS errors; an emulated browser position in London produced “London, United Kingdom” via the geocoder, cached it under soupcon_geocode_, and saved soupcon_last_source=browser, while an emulated Seattle position used NWS; a 390 px wide page loads. **Found and fixed by this check:** a non-US lookup sent an NWS alerts request that could only fail (HTTP 400, a console error); alerts are now requested only after NWS confirms coverage (no added latency, the forecast requests wait on the same gridpoint lookup and take longer), with a test that no alerts request is sent for a 404 point. The one remaining console line for a non-US lookup is the browser logging the NWS coverage probe’s 404, which can’t be suppressed. **Not verified:** a rain or snow case in the browser (none was forecast for any checked place at the time; those paths are unit-tested with fixtures only), the PWA install flow, print, and a real phone; the service worker was not involved (preview build, no install).

Item 22 is complete. Open follow-ups, all optional: verify a snow forecast live when one exists, and decide whether ?lat=&lon= places should get a name without the geocoder (see step 5).

- **Explicitly out of scope:** non-US ZIP/postal-code lookup (would need per-country validation; lat/lon and browser location are the way in), and any non-US alerts source.

23. Clearer browser-geolocation errors — FIXED

- **Report (from a friend in Romania):** the app showed “Timeout expired Try entering a US ZIP code instead.” Two problems: a non-US visitor was told to enter a US ZIP, and “Timeout expired” is the browser’s own bare message text (Chrome’s wording for a geolocation timeout, not an HTTP timeout), which says nothing actionable. The lookup waits 15 s for a quick low-accuracy fix, then up to 30 s more for a high-accuracy retry, so the visitor had waited about 45 seconds for the device to report its position.
- **Done:** new `src/lib/geolocationError.js` with `geolocationErrorMessage(geoError)`, which writes plain-language text per error code instead of passing the browser’s message through: a timeout says the device did not report its location within 45 seconds and to check location services; position-unavailable says the device could not work out where it is (check location services and signal); permission-denied keeps the existing iPhone settings instructions; anything else is a generic “couldn’t read your location”. Every message ends with `ZIP_FALLBACK_HINT`, “If you’re in the US, you can enter a ZIP code instead.”, replacing the unconditional “Try entering a US ZIP code”. The “browser does not support geolocation” message uses the same hint. The two timeouts are now named constants (`GEOLLOCATION_FAST_TIMEOUT_MS`, `GEOLLOCATION_PRECISE_TIMEOUT_MS`) in that module, used by `App.jsx`, so the “45 seconds” in the text cannot drift from the real wait. 4 new tests; 130/130 tests, lint and build pass. Checked in headless Chrome with a stubbed `navigator.geolocation` failing with codes 3, 2 and 1: each showed the expected message. Not reproduced for real (the cause on the friend’s device, such as location services off or a desktop with no GPS, is unknown).
- **Not done, raised for the owner:** a non-US visitor whose device can’t report a location still has no way in except the `lat/lon` URL parameters. A place-name search would fix that: Open-Meteo’s geocoding API searches by name (free, no key), unlike the reverse lookup we can’t use. Not built; a product call.
- **Follow-up, reverted (per owner): the high-accuracy retry stays.** The owner asked whether the retry could go (low accuracy only), then decided to keep it because low accuracy does not always work, and the retry is what rescues those cases. It was briefly removed and is restored exactly as before: a low-accuracy try (15 s, `maximumAge` 10 min), then, for any failure except permission-denied, one high-accuracy retry (30 s, no cached position) with the “Still locating you... trying a more precise lookup.” message. `GEOLLOCATION_FAST_TIMEOUT_MS` and `GEOLLOCATION_PRECISE_TIMEOUT_MS` are back, and the timeout message again says 45 seconds. The cost, knowingly accepted, is that a lookup that genuinely fails takes about 45 seconds to report it. **Don’t re-raise removing it without new information.**

24. Country dropdown and city search — FIXED

- **Ask (per owner):** add a dropdown to the right of the ZIP field marked “Country”, defaulting to “USA”; if it is changed, replace the ZIP field with a text field and the button with a search button that searches that country for that city. (Motivated by item 23: a visitor outside the US whose device can’t report a location had no way in except `lat/lon` URL parameters.)
- **Probed first** (`geocoding-api.open-meteo.com/v1/search`): free, no key, CORS open; `countryCode` (ISO alpha-2, case-insensitive) limits the search; results are ranked, largest places first, with `name`, `admin1`, `country`, `latitude`, `longitude`, `feature_code`; a name with no match omits results entirely; a 1-character query returns nothing. A name can return several matches (real case: “Cluj” gave the city and its airport; “San Juan” gave 9 incl. a steam-plant heliport), so a pick list is needed. Forward geocoding (name to place) has none of the reverse geocoder’s device-location restriction from item 5.
- **Done:**
 - **UI (`App.jsx`, `styles.css`):** the ZIP/city field, then a labeled Country select, then the button. USA (default, listed first as “USA”) keeps the ZIP field and “Search US ZIP” exactly as before; any other country shows a “City” field (Enter city) and “Search city” (busy text “Looking up...”). Switching country clears any pick list and error. A query under 2 letters gets “Enter at least 2 letters of a city name.”; no match clears the result and says “We couldn’t find “x” in Romania. Check the spelling, or try a larger nearby city.”; one match is looked up directly; several are shown as a “Which one?” list of buttons. Stale-request guards

- mirror the ZIP search (sequence counter + AbortController).
- **src/lib/placeSearch.js and src/data/countryCodes.js:** `getCountryOptions()` (250 codes, names from `Intl.DisplayNames` so there is no hand-kept name table; USA first, the rest alphabetical), `countryName`, `formatSearchResultLabel` ("Cluj-Napoca, Cluj County, Romania"; a repeated part is dropped, so Bucharest reads "Bucharest, Romania") and `searchPlaces(query, countryCode)`. The code list is ISO 3166-1 alpha-2 plus XK (Kosovo, which the geocoder accepts), written by hand because the runtime's own list includes obsolete codes (UK, YU, ...); a test guards against duplicates and obsolete codes. **Populated places only:** asks for 20 results and keeps up to 8 whose `feature_code` starts with PPL, falling back to everything if none do, so airports and heliports do not show as cities.
 - **Lookup wiring:** `lookupWeather` takes an optional label (the place's own name) used as the Open-Meteo place name, bypassing reverse geocoding; source "city". The pick is remembered (`soupcon_last_source=city`, `soupcon_last_place` with `lat/lon/label/country/query`) only after it succeeds and resumed on the next visit from those saved coordinates (Country and city field restored). URL coordinates still win over a remembered lookup. A country NWS covers (Puerto Rico verified) still routes to NWS and keeps NWS's own "San Juan, PR" label and alerts.
 - **Also changed:** the geolocation-failure hint (item 23) now ends "...otherwise choose your country and search for your city." so the error points at this feature.
 - **Tests:** `placeSearch.test.js` (13: options, labels, request URL, the populated-place filter and its fallback, the 8-result cap, empty result, bad coordinates, HTTP failure) and a weatherProvider test for the label override; 143/143 tests, lint and build pass.
 - **Verified in headless Chrome** (real APIs, production build): the default form is unchanged (USA selected, 250 options, USA first); choosing Romania swaps ZIP for City and "Search city"; 1 letter gives the error; "Cluj" (before the filter) showed the two-choice list and picking the city gave "#SOUPCON4 CONDITION: Cluj-Napoca, Cluj County, Romania ..." with no alerts section; the pick was saved and a reload resumed it with Romania and "Cluj" restored; "zzzzq" gave the not-found message and cleared the result; switching back to USA restored the ZIP field and a ZIP search worked; Puerto Rico "San Juan" gave a list, picking one routed to NWS ("San Juan, PR", alert count shown); phone width (390 px) wraps the row cleanly; no JavaScript errors. **Re-checked in the browser after the populated-place filter:** "Cluj" now resolves to one place and goes straight to the forecast with no pick list (the airport is filtered out); "San Juan" in Puerto Rico narrows from 9 results to 5.
 - **Known limits:** US cities are not searchable (ZIP is the US path); a search returns at most 8 places and uses the geocoder's ranking, so a small town with a famous namesake may need its region added to the query; other-language and accent handling is whatever the geocoder does (diacritics matched in a probe: "Ploiesti" found "Ploiești").

25. Review of items 19-24 against the code — FIXED

- **Ask (per owner, 2026-09-30):** review the plan and the current codebase and make sure there are no flaws. Read every source module, ran the suite (143/143), lint and build (all clean) before changing anything.
- **Done (small fixes):**
 - **Stale "Which one?" list.** It stayed on screen after clicking "Use Browser Location", and a city search still in flight when the Country dropdown changed could land its matches afterwards (e.g. Romanian places listed under the US ZIP field). `handleUseBrowserLocation` now clears the list, and the list only renders when its places were searched in the currently selected country. Deliberately not done by aborting the search on a country change: the stale-request paths leave loading to the newer lookup, so aborting with no newer lookup would leave the buttons disabled.
 - **Sources panel wording (Open-Meteo):** said "There are no weather stations outside the US", which isn't true; now "Outside the US there are no station readings; Open-Meteo's own model estimate decides level 1."
 - **Docs drift:** item 22 above was still headed "Plan only, nothing built yet" (now marked done); the Status section below was dated after item 18; `CONTEXT.md` and a `weatherApi.js` comment said the four NWS calls run concurrently (since item 22 `lookupWeather` awaits the

- label/coverage probe first); README.md/CONTEXT.md described the remembered method as browser-or-ZIP only (city search too now); public/index.md didn't list the geocoding APIs.
- **Found:** the soup of the day was picked once on mount, so an installed PWA left running across midnight kept yesterday's pick until reloaded. **Fixed per owner** — see item 19's follow-up.
 - **Found:** runLookupFromCoordinates/lookupWeather's skipCache option was never passed as true (and on the NWS path only reached alerts). **Removed per owner** from runLookupFromCoordinates, lookupWeather and lookupOpenMeteo; the per-source API functions keep theirs, since refreshWeather uses it. Step 6's note in item 22 describes the original signature. Lint, 145/145 tests and build pass.
 - Lint, 143/143 tests and build pass. The UI changes are JSX conditions only, not checked in a browser.
-

Suggested order

1. **#1 + #2** (classification + API layer — the real architectural change)
2. **#3** (wire into App.jsx, resolving the alerts-panel question)
3. **#4 + #5 + #6** (content, component renames, CSS renames — the app now *works* and *looks* like SOUPCON end-to-end)
4. **#11** (tests, while the classification logic is still fresh)
5. **#7 + #8 + #9** (branding assets, domain-referencing static files, storage key renames — cosmetic/polish, safe to batch)
6. **#10** (docs rewrite, once there's a stable thing to document)
7. **#12 + #13** (infra + launch — last, and gated on domain registration)

Status (updated 2026-09-30, after item 25)

The rebuild (items 1-11) and infra/launch (12-13, done by the owner) are complete and soupcon.org is live. Items 14-25 are post-launch fixes and features (including worldwide weather via Open-Meteo, item 22, and city search, item 24), all done. What's left:

- **Worldwide follow-ups (optional, from item 22):** check a real snow forecast in the Open-Meteo path when one exists (the snow codes are unit-tested only), and decide whether ?lat=&lon= places should get a name without the reverse geocoder.
- **Real-device verification** — the app was finally exercised in a real (headless) browser in item 17: lookups, overlays, recipe modal, share, snow pill, phone width. Still unverified: the PWA install flow on an actual phone (only testable from the production root, see CONTEXT.md), and an actual print preview of the recipe.
- **Cloudflare's managed robots.txt** shadowing the explicit ai-input=yes stance — a real, open decision (see CONTEXT.md), not a bug to fix.
- **Deliberately left as-is (owner decisions, don't re-raise):** an untyped precipitationLastHour reading defaults to rain; the 90-minute observation age limit; NWS alert descriptions' hard line breaks.